Programmation Impérative 3: Initiation à l'algorithmique

Cours 6

Liste, Pile et File

Pierre Fouilhoux
pierre.fouilhoux@lipn.fr

1 septembre 2025

1. Structures de données abstraites (SDA)
2. SDA Liste
3. Implementation d'une liste par un Tableau
4. Implementation d'une liste par une Liste Simplement Chaînée
5. Pile et File
6. Pile d'éxecution et Dérécursification

Structure de données abstraites (SDA)

Structure de Données (SD)

Ensemble organisé d'éléments situés dans la mémoire de l'ordinateur et possédant des opérations de construction, d'accès, de modification et de suppression.

- Les éléments constituant une SD sont repérés dans un programme par des variables (typées) ou, récursivement, par une SD.
- Certaines SD sont directement définies dans les langages :
  - C possède de base les tableaux à une dimension et les types utilisateurs (les struct).
  - Python, Java ou C++ possède des SD plus avancées comme les listes, les listes-tableaux, les maps...
- Certaines SD n'existent que sur certains types de machines.
  Par exemple, les banques de mémoire vectorielle qui n'existent que sur les machines vectorielles.

Structure de Données Abstraite

Les SD les plus utilisées sont définies de manière à s'abstraire de la machine et du langage.

Structure de Données Abstraites (SDA)

Description abstraite d'une Structure de Données indiquant :
- son organisation des données
- son *interface*, c'est-à-dire ses méthodes de manipulation.

Une interface d'une SDA contient nécessairement :

- déclaration et allocation
- accès à un élément
- ajout d'un élément
- modification d'un élément
- suppression et désallocation d'un élément
- désallocation de la SDA

Structure de Données Abstraite

• Une interface d'une SDA peut contenir aussi des méthodes plus spécifiques comme la recherche d'un élément, le tri...

• Toutes les SDA n'ont pas les mêmes

- *cas d'utilisation*
- ni les même *performances*

qui dépendent de l'implémentations, du langage choisi et parfois des machines.

• On peut aussi leur caractériser certains caractéristiques techniques comme
- la généricité
- le polymorphisme
- l'hétérogénïté

Complexité et occupation mémoire d'une SDA

Et bien-entendu, toutes les implémentations d'une SDA ne sont pas identiques concernant leurs performances.

- **complexité-temps** : Vitesse des fonctions de manipulation
- **complexité-espace** : occupation mémoire d'une SDA

1 Structures de données abstraites (SDA)

2 SDA Liste

3 Implementation d'une liste par un Tableau

4 Implementation d'une liste par une Liste Simplement Chaînée

5 Pile et File

6 Pile d'éxecution et Dérécursification

SDA Liste

Une Liste (linéaire) est une Structure de Données Abstraites (SDA) correspondant

- à une suite finie $(elnt, elnt, elnt, elnt, \ldots, elnt)$ d'éléments.
- où l'on peut passer d'un élément à l'élément suivant par une fonction dite `successeur`.

La liste est *vide* si elle ne contient aucun élément.

Une liste non vide a deux extrémités que l'on appelle *début* (ou tête) et *fin* (ou queue).

SDA Liste

Cas d'utilisation

Une liste peut représenter

- dans la vie courante tous les principes de files d'attente : attente en magasins, dans une liste administrative de traitements de dossiers...
- dans un système informatique, c'est la gestion dans l'ordre de lancement des processus (de même priorité) qui seront traités l'un après l'autre
- dans un logiciel, c'est une structure très fréquente et utile. Un exemple parlant est la liste des actions faites dans un traitement de texte : on peut revenir en arriète (et parfois revenir ensuite en avant) dans l'ordre (inverse) des actions. Les actions sont ainsi stockées dans une liste, où Undo (CTRL+Z), fait remonter à la dernière, puis l'avant-dernière etc des actions faites.

SDA Liste
Complexités pour la liste simplement chaînée

Pour une Liste *L* contenant *n* éléments.
On manipule *it* un itérateur “pointant” un élément de la liste.

Successeur(*L*, *it***)** : retourne l’itérateur successeur de *it*

Accès_deb(*L***) :** retourne l’itérateur sur l’élément de début.
Accès_fin(*L***) :** retourne l’itérateur accès à l’élément de fin.

Suppression_successeur(*L*, *it***)** : supprime l’élément successeur de *it*
Insertion_debut(*L*, *elmt***)** : insère un élément en début de liste
Insertion_debut(*L*, *elmt***)** : insère un élément en début de liste

Et parfois
Accès(*L*, *i***)** : accès à l’élément *i*
Prédécesseur(*L*, *it***)** : retourne l’itérateur successeur de *it*
Recherche(*L*, *elnt***)** : recherche si l’élément *elnt* est dans la liste et retourne un itérateur dessus si c’est le cas

SDA Liste
Complexités pour la liste simplement chaînée

Pour une Liste *L* contenant *n* éléments.
On manipule *it* un itérateur “pointant” un élément de la liste.

Successeur(*L*, *it***)** : retourne l’itérateur successeur de *it*

Accès_deb(*L***) :** retourne l’itérateur sur l’élément de début.
Accès_fin(*L***) :** retourne l’itérateur accès à l’élément de fin.

Suppression_successeur(*L*, *it***)** : supprime l’élément successeur de *it*
Insertion_debut(*L*, *elmt***)** : insère un élément en début de liste
Insertion_debut(*L*, *elmt***)** : insère un élément en début de liste

Et parfois
Accès(*L*, *i***)** : accès à l’élément *i*
Prédécesseur(*L*, *it***)** : retourne l’itérateur successeur de *it*
Recherche(*L*, *elnt***)** : recherche si l’élément *elnt* est dans la liste et retourne un itérateur dessus si c’est le cas

SDA Liste
Complexités pour la liste simplement chaînée

Pour une Liste *L* contenant *n* éléments.
On manipule *it* un itérateur “pointant” un élément de la liste.

Successeur(*L*, *it***)** : retourne l’itérateur successeur de *it*

Accès_deb(*L***) :** retourne l’itérateur sur l’élément de début.
Accès_fin(*L***) :** retourne l’itérateur accès à l’élément de fin.

Suppression_successeur(*L*, *it***)** : supprime l’élément successeur de *it*
Insertion_debut(*L*, *elmt***)** : insère un élément en début de liste
Insertion_debut(*L*, *elmt***)** : insère un élément en début de liste

Et parfois
Accès(*L*, *i***)** : accès à l’élément *i*
Prédécesseur(*L*, *it***)** : retourne l’itérateur successeur de *it*
Recherche(*L*, *elnt***)** : recherche si l’élément *elnt* est dans la liste et retourne un itérateur dessus si c’est le cas

SDA Liste
Complexités pour la liste simplement chaînée

Pour une Liste L contenant n éléments.
On manipule *it* un itérateur “pointant” un élément de la liste.

Successeur(L, *it***)** : retourne l’itérateur successeur de *it*

Accès_deb(L**) :** retourne l’itérateur sur l’élément de début.
Accès_fin(L**) :** retourne l’itérateur accès à l’élément de fin.

Suppression_successeur(L, *it***)** : supprime l’élément successeur de *it*
Insertion_debut(L, *elmt***)** : insère un élément en début de liste
Insertion_debut(L, *elmt***)** : insère un élément en début de liste

Et parfois
Accès(L, i**)** : accès à l’élément i
Prédécesseur(L, *it***)** : retourne l’itérateur successeur de *it*
Recherche(L, *elnt***)** : recherche si l’élément *elnt* est dans la liste et retourne un itérateur dessus si c’est le cas

SDA Liste
Complexités pour la liste simplement chaînée

Pour une Liste *L* contenant *n* éléments.
On manipule *it* un itérateur “pointant” un élément de la liste.

Successeur(*L*, *it***)** : retourne l’itérateur successeur de *it*

Accès_deb(*L***) :** retourne l’itérateur sur l’élément de début.
Accès_fin(*L***) :** retourne l’itérateur accès à l’élément de fin.

Suppression_successeur(*L*, *it***)** : supprime l’élément successeur de *it*
Insertion_debut(*L*, *elmt***)** : insère un élément en début de liste
Insertion_debut(*L*, *elmt***)** : insère un élément en début de liste

Et parfois
Accès(*L*, *i***)** : accès à l’élément *i*
Prédécesseur(*L*, *it***)** : retourne l’itérateur successeur de *it*
Recherche(*L*, *elnt***)** : recherche si l’élément *elnt* est dans la liste et retourne un itérateur dessus si c’est le cas

SDA Liste

On peut coder une liste

- par un tableau à 1 dimension.
- mais nous allons voir qu'on prévilégie une implémentation par une Liste Chaînée.

1 Structures de données abstraites (SDA)

2 SDA Liste

3 Implementation d'une liste par un Tableau

4 Implementation d'une liste par une Liste Simplement Chaînée

5 Pile et File

6 Pile d'éxecution et Dérécursification

Allocation dynamique

Dans la mémoire (sur le tas), on sait

- réserver de l'espace mémoire,
- “modifier la taille d'un espace mémoire”,
- libérer cet espace.

Cela permet de créer des tableaux ≪ à la volée ≫,
en ajustant leur taille selon le besoin.

Implémentations par un tableau

Voici un exemple d'implémentation d'une SDA liste par un tableau

```
1 typedef struct{
2   char * tab;      /* Tableau dont les éléments sont des char */
3   int n;           /* Nombre d'éléments du tableau (taille logique) */
4   int MAX_SIZE;    /* Taille physique du tableau */
5 } Liste_par_un_Tableau;
```

Remarques pour cette implémentation

- On encapsule un tableau dans `Liste_par_un_Tableau` , c'est-à-dire qu'on "cache" la logique du code derrière une entrée facile à gérer dans le `main`.
- Comme tout tableau, il y a une taille physique à donner pour l'allocation.

Implémentation par un tableau

```
1 Liste_par_un_Tableau * creer_liste_par_un_tableau(int max_size){
2   Liste_par_un_Tableau *LT;
3   LT= malloc(sizeof(Liste_par_un_Tableau));
4   LT->tab=malloc(max_size*sizeof(char));
5   if (LT->tab==NULL){
6     printf("ERREUR: mémoire pleine\n");
7     exit(2);
8   }
9   LT->MAX_SIZE=max_size;
0   LT->n=0;
1 }
```

Rq : Tester si le premier `malloc` a bien fonctionné a peu d'utilité étant donné la petite taille de ce qui est réservé.

Implémentation par un tableau

```
1 int test_liste_vide(Liste_par_un_Tableau *LT){
2   return LT->n==0;
3 }
```

```
1 void afficher_liste(Liste_par_un_Tableau *LT){
2   int i;
3   if (LT->n==0){
4     printf("Liste vide\n");
5   }
6   else{
7     printf("( %c", LT->tab[0]);
8     for (i=1;i<LT->n;i++)
9       printf(", %c",LT->tab[i]);
0     printf(" )\n");
1   }
2 }
```

Rq : On peut utiliser la fonction `test_liste_vide` en ligne 3, mais c'est une perte d'efficacité. Cette fonction est faite pour les utilisateurs de `Liste_par_un_Tableau`.

Implémentation par un tableau

```
1 void affiche_debut(Liste_par_un_Tableau *LT){
2   if (LT->n==0)
3     printf("Liste vide\n");
4   else
5     printf("Element au debut: %c\n",LT->tab[0]);
6 }
```

```
1 void affiche_fin(Liste_par_un_Tableau *LT){
2   if (LT->n==0)
3     printf("Liste vide\n");
4   else
5     printf("Element en fin: %c\n", LT->tab[LT->n-1]);
6 }
```

```
1 void affiche_elnt_i(Liste_par_un_Tableau *LT, int i){
2   if (i<LT->n)
3     printf("Element inconnu\n");
4   else
5     printf("Element %d : %c\n", i, LT->tab[i]);
6 }
```

Complexité : en $O(1)$ grâce à la nature indexée d'un tableau.

Implémentation par un tableau

```
1 void ajouter_fin(Liste_par_un_Tableau *LT, char elnt){
2   if (LT->n==LT->MAX_SIZE){
3     printf("Liste pleine\n");
4   }
5   else{
6     LT->tab[LT->n]=elnt;
7     LT->n++;
8   }
9 }
```

Complexité : Insérer en fin est seulement en $O(1)$.

Implémentation par un tableau

```
1 void ajouter_debut(Liste_par_un_Tableau *LT, char elnt){
2   int i;
3   if (LT->n==LT->MAX_SIZE){
4     printf("Liste pleine\n");
5   }
6   else{
7     for(i=LT->n;i>=1;i--)
8       LT->tab[i]=LT->tab[i-1];
9     LT->tab[0]=elnt;
0     LT->n++;
1   }
2 }
```

Complexité : Mais insérer en fin est en $O(n)$...

et insérer en position i est en $O(i)$.

Implémentation par un tableau

```
1 void ajouter_debut(Liste_par_un_Tableau *LT, char elnt){
2   int i;
3   if (LT->n==LT->MAX_SIZE){
4     printf("Liste pleine\n");
5   }
6   else{
7     for(i=LT->n;i>=1;i--)
8       LT->tab[i]=LT->tab[i-1];
9     LT->tab[0]=elnt;
0     LT->n++;
1   }
2 }
```

Complexité : Mais insérer en fin est en $O(n)$...

et insérer en position i est en $O(i)$.

Implémentation par un tableau

```
1 void ajouter_debut(Liste_par_un_Tableau *LT, char elnt){
2   int i;
3   if (LT->n==LT->MAX_SIZE){
4     printf("Liste pleine\n");
5   }
6   else{
7     for(i=LT->n;i>=1;i--)
8       LT->tab[i]=LT->tab[i-1];
9     LT->tab[0]=elnt;
0     LT->n++;
1   }
2 }
```

Complexité : Mais insérer en fin est en $O(n)$...

et insérer en position i est en $O(i)$.

Implémentation par un tableau

```
1 int main(){
2
3   Liste_par_un_Tableau *LT=creer_liste_par_un_tableau(10);
4   ajouter_fin(LT,'c');
5     afficher_liste(LT);
6   ajouter_fin(LT,'e');
7     afficher_liste(LT);
8   ajouter_debut(LT,'p');
9     afficher_liste(LT);
0
1   detruire_liste(&LT);
2
3   return EXIT_SUCCESS;
4 }
```

```
( c )
( c, e )
( p, c, e )
```

Implémentation par un tableau

```
1 void detruire_liste(Liste_par_un_Tableau **LT){
2   free((*LT)->tab);
3   free(*LT);
4   *LT=NULL;
5 }
```

On fait passer l'adresse d'un pointeur sur `Liste_par_un_Tableau` afin de le mettre à NULL dans la fonction de désallocation.

Implémentation par un tableau

Comment ajouter un élément au-delà de la taille physique ?

```
1 void ajouter_fin_avec_realloc(Liste_par_un_Tableau *LT, char elnt){
2   if (LT->n==LT->MAX_SIZE){
3     LT->MAX_SIZE+=10;
4     LT->tab=realloc(LT->tab,LT->MAX_SIZE*sizeof(char));
5     if (LT->tab==NULL){
6       printf("ERREUR: Mémoire pleine\n");
7       exit(2);
8     }
9   }
0   LT->tab[LT->n]=elnt;
1   LT->n++;
2 }
```

Rq : On augmente, dans cet exemple,la taille du tableau de 10 cases supplémentaires, mais cela pourraît être 1 ou 100 : cette quantité dépend du contexte d'usage de la liste.

Complexité : ?

Implémentation par un tableau

Comment ajouter un élément au-delà de la taille physique ?

```
1  void ajouter_fin_avec_realloc(Liste_par_un_Tableau *LT, char elnt){
2    if (LT->n==LT->MAX_SIZE){
3      LT->MAX_SIZE+=10;
4      LT->tab=realloc(LT->tab,LT->MAX_SIZE*sizeof(char));
5      if (LT->tab==NULL){
6        printf("ERREUR: Mémoire pleine\n");
7        exit(2);
8      }
9    }
0    LT->tab[LT->n]=elnt;
1    LT->n++;
2  }
```

Rq : On augmente, dans cet exemple,la taille du tableau de 10 cases supplémentaires, mais cela pourraît être 1 ou 100 : cette quantité dépend du contexte d'usage de la liste.

Complexité : ?

Implémentation par un tableau

Comment ajouter un élément au-delà de la taille physique ?

```
1  void ajouter_fin_avec_realloc(Liste_par_un_Tableau *LT, char elnt){
2    if (LT->n==LT->MAX_SIZE){
3      LT->MAX_SIZE+=10;
4      LT->tab=realloc(LT->tab,LT->MAX_SIZE*sizeof(char));
5      if (LT->tab==NULL){
6        printf("ERREUR: Mémoire pleine\n");
7        exit(2);
8      }
9    }
0    LT->tab[LT->n]=elnt;
1    LT->n++;
2  }
```

Rq : On augmente, dans cet exemple,la taille du tableau de 10 cases supplémentaires, mais cela pourraît être 1 ou 100 : cette quantité dépend du contexte d'usage de la liste.

Complexité : ?

Le problème de la taille supplémentaire

De combien de cases réallouer ?

Lorsqu'un tableau est plein et que l'on souhaite réallouer

- Soit on décide de réserver juste une seule case.
  - La quantité de mémoire sera toujours adaptée au besoin.
  - Mais à chaque fois, on devra recopier tout le tableau.
- Soit on réserve plusieurs cases.
  - On risque de réserver plus d'espace que nécessaire.
  - Les ajouts suivants seront rapides.

Le problème de l'espace supplémentaire

Demander de l'espace supplémentaire

La fonction `realloc` d'un tableau de n cases, a une complexité en $O(n)$

On veut ajouter la valeur 3 au tableau

6	8	3	8	4	12	5	1

On crée un tableau plus grand.

?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?

On recopie les n cases du petit tableau dans le grand.

6	8	3	8	4	12	5	1	?	?	?	?	?	?	?	?

On ajoute la valeur 3.

6	8	3	8	4	12	5	1	3	?	?	?	?	?	?	?

Le problème de l'espace supplémentaire

Demander de l'espace supplémentaire

La fonction `realloc` d'un tableau de n cases, a une complexité en $O(n)$

On veut ajouter la valeur 3 au tableau

6	8	3	8	4	12	5	1

On crée un tableau plus grand.

?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?

On recopie les n cases du petit tableau dans le grand.

6	8	3	8	4	12	5	1	?	?	?	?	?	?	?	?

On ajoute la valeur 3.

6	8	3	8	4	12	5	1	3	?	?	?	?	?	?	?

Le problème de l'espace supplémentaire

Demander de l'espace supplémentaire

La fonction `realloc` d'un tableau de n cases, a une complexité en $O(n)$

On veut ajouter la valeur 3 au tableau

6	8	3	8	4	12	5	1

On crée un tableau plus grand.

?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?

On recopie les n cases du petit tableau dans le grand.

6	8	3	8	4	12	5	1	?	?	?	?	?	?	?	?

On ajoute la valeur 3.

6	8	3	8	4	12	5	1	3	?	?	?	?	?	?	?

Le problème de l'espace supplémentaire

Demander de l'espace supplémentaire

La fonction `realloc` d'un tableau de n cases, a une complexité en $O(n)$

On veut ajouter la valeur 3 au tableau

6	8	3	8	4	12	5	1

On crée un tableau plus grand.

?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?

On recopie les n cases du petit tableau dans le grand.

6	8	3	8	4	12	5	1	?	?	?	?	?	?	?	?

On ajoute la valeur 3.

6	8	3	8	4	12	5	1	3	?	?	?	?	?	?	?

Implémentation par un tableau

Manipulation

Pour une SDA liste implémentée par un tableau :

- Initialiser : demande de fixer une taille initiale au tableau. (cela peut réserver trop ou pas assez de places).
- Détruire : $O(1)$
- Tester si un tableau est vide : $O(1)$
- Connaître le nombre d'éléments dans le tableau : $O(1)$
- Ajouter un élément :
  - $O(1)$ si le tableau n'est pas plein
  - $O(n)$ si le tableau est plein.
- Supprimer un élément : $O(n)$

1 Structures de données abstraites (SDA)

2 SDA Liste

3 Implementation d'une liste par un Tableau

4 Implementation d'une liste par une Liste Simplement Chaînée

5 Pile et File

6 Pile d'éxecution et Dérécursification

Les listes chaînées

L'idée

- On veut une structure de données que l'on peut agrandir en temps constant.
- Et donc ajouter juste une case mémoire à chaque étape.

En revanche

- On va voir que si certaines fonctions deviennent plus rapides, d'autres deviennent plus lentes.

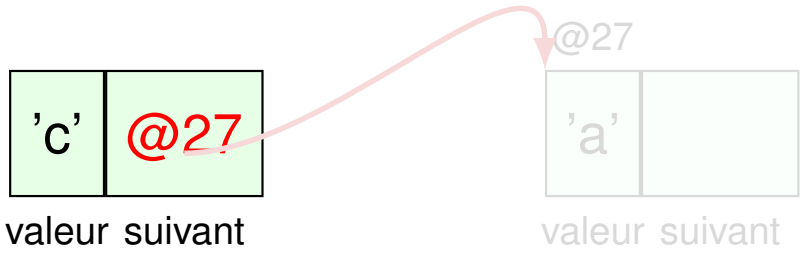
Les listes chaînées : les maillons

L'idée : les maillons

Un `maillon` (ou `Cellule`) d'une liste est constitué de :

- une valeur (d'un type scalaire ou autres (struct...))
- un pointeur vers le maillon suivant.

Exemple : Deux maillons contenant des données de type `char`.



Ici le maillon de gauche contient l'adresse du maillon de droite dans le pointeur suivant.
On appelle cela le chaînage.

Les listes chaînées : les maillons

L'idée : les maillons

Un `maillon` (ou `Cellule`) d'une liste est constitué de :

- une valeur (d'un type scalaire ou autres (struct...))
- un pointeur vers le maillon suivant.

Exemple : Deux maillons contenant des données de type `char`.

Ici le maillon de gauche contient l'adresse du maillon de droite dans le pointeur suivant.
On appelle cela le chaînage.

Les maillons

```
1 struct maillon_s {
2   char valeur;                /* Données stockées */
3   struct maillon_s * suivant; /* Pointeur sur suivant */
4 };
5
6 typedef struct maillon_s Maillon;
```

Remarque

C'est une forme de définition "récursive", mais il s'agit simplement de la réservation d'espace mémoire pour un struct à deux champs !

Les maillons : création et initialisation ; et destruction

- Fonction qui crée un maillon, initialise sa valeur à `val`, et renvoie l'adresse du maillon ainsi créé.

```
1 Maillon * creer_maillon (char val) {
2   Maillon * res = malloc(sizeof(Maillon));
3   res->valeur = val;
4   res->suivant = NULL;
5   return res;
6 }
```

Ici, on ne teste pas la mémoire, ce n'est que l'ajout d'une toute petite quantité de mémoire : si la mémoire est saturée au point de faire un stack overflow pour cela, le programme va s'arrêter...

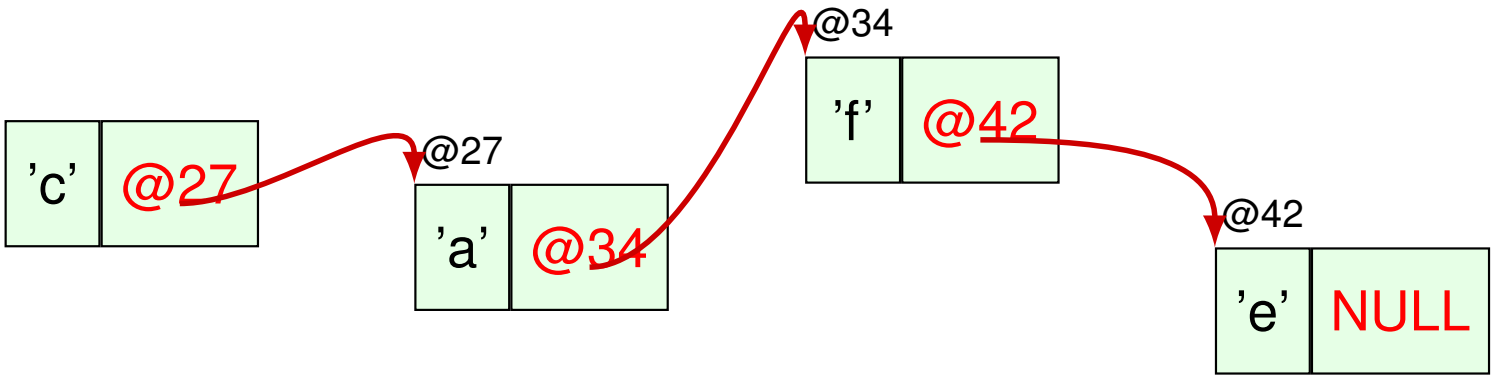
- Fonction qui libère toute la memoire occupèe par le maillon dont l'adresse est passée en paramètre.

```
1 void detruire_maillon (Maillon * m) {
2   free(m);
3 }
```

La liste simplement chaînée

Une liste simplement chaînée est une structure de donnée définie par un chaînage linéaire unidirectionnel.

C'est-à-dire un ensemble de maillons contenant chacun l'adresse d'un autre maillon de manière à former une sorte de "ligne" tirée virtuellement entre des cases mémoire éparpillée.



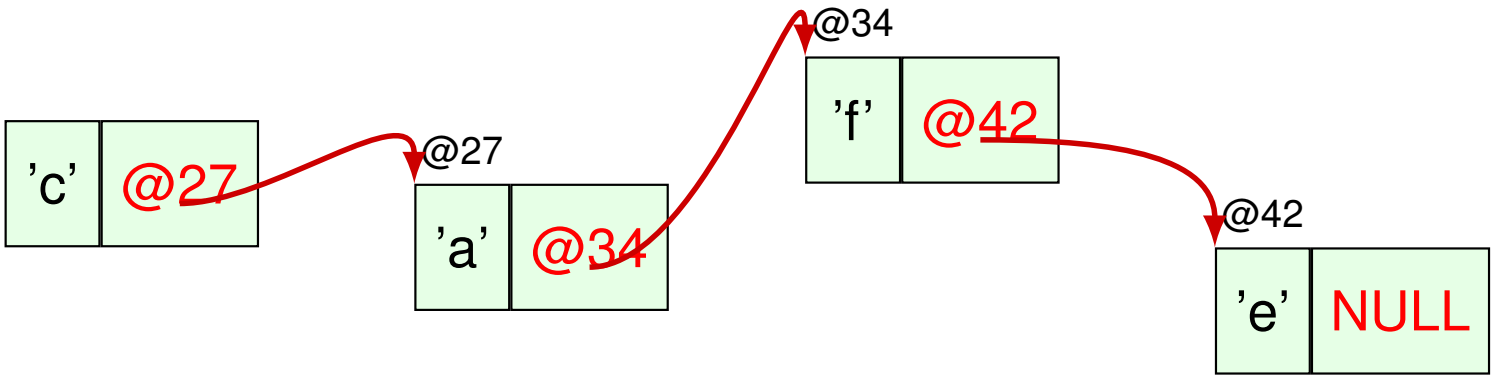
On rappelle que la flèche sortant d'un pointeur désigne simplement le fait qu'un pointeur contient l'adresse d'une autre case mémoire.

La dernier maillon de la liste chaînée contient le pointeur NULL.

La liste simplement chaînée

Une liste simplement chaînée est une structure de donnée définie par un chaînage linéaire unidirectionnel.

C'est-à-dire un ensemble de maillons contenant chacun l'adresse d'un autre maillon de manière à former une sorte de "ligne" tirée virtuellement entre des cases mémoire éparpillée.



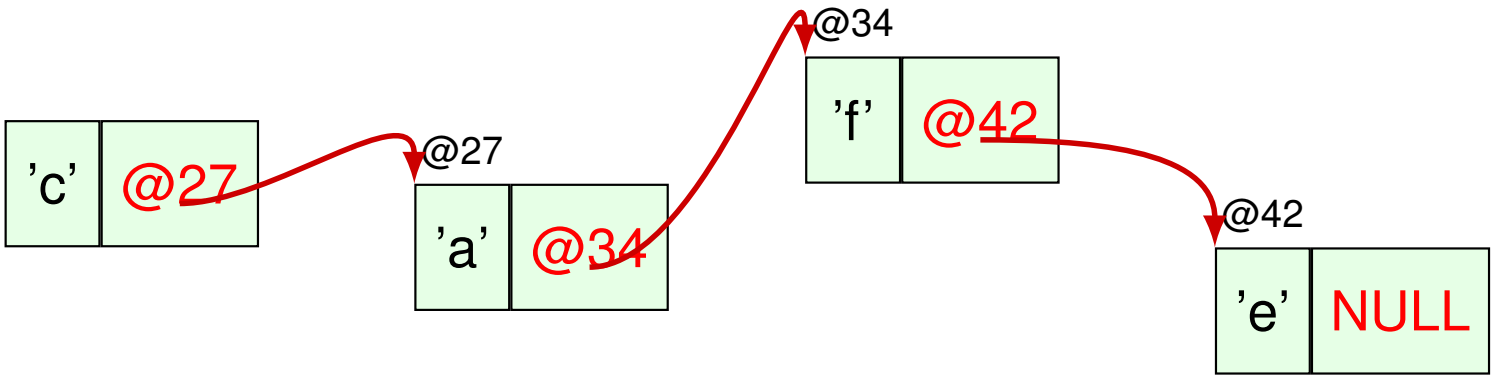
On rappelle que la flèche sortant d'un pointeur désigne simplement le fait qu'un pointeur contient l'adresse d'une autre case mémoire.

La dernier maillon de la liste chaînée contient le pointeur NULL.

La liste simplement chaînée

Une liste simplement chaînée est une structure de donnée définie par un chaînage linéaire unidirectionnel.

C'est-à-dire un ensemble de maillons contenant chacun l'adresse d'un autre maillon de manière à former une sorte de "ligne" tirée virtuellement entre des cases mémoire éparpillée.



On rappelle que la flèche sortant d'un pointeur désigne simplement le fait qu'un pointeur contient l'adresse d'une autre case mémoire.

La dernier maillon de la liste chaînée contient le pointeur NULL.

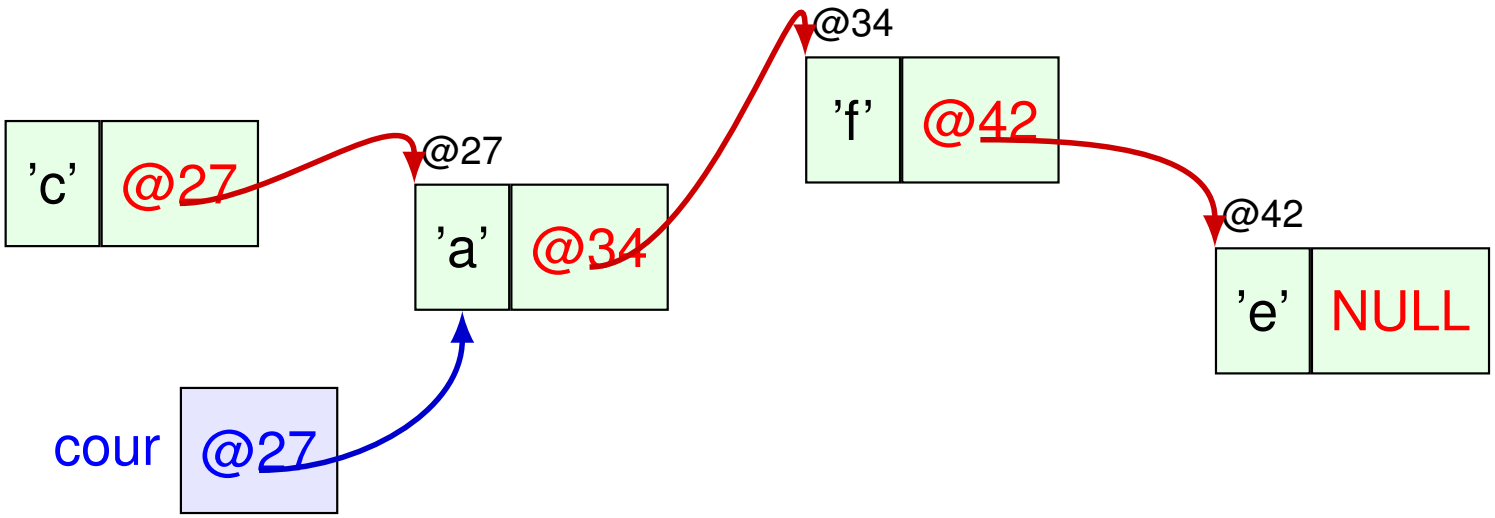
Itérateur et Fonction successeur

- Un itérateur est une variable permettant de passer d'élément en élément. Ici c'est un pointeur courant

```
maillon *cour = @27;
```

- Le fait de passer d'un maillon à un autre est la fonction successeur.

```
cour=cour->suivant;   // Passe à @42
```



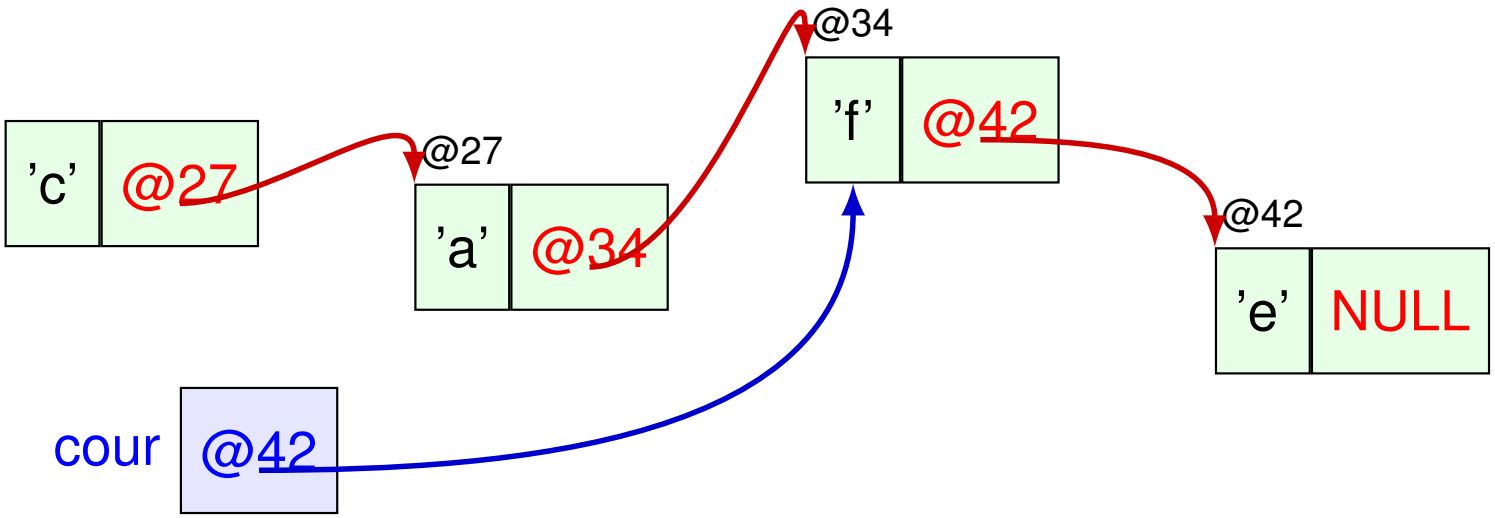
Itérateur et Fonction successeur

- Un itérateur est une variable permettant de passer d'élément en élément. Ici c'est un pointeur courant

```
maillon *cour = @27;
```

- Le fait de passer d'un maillon à un autre est la fonction successeur.

```
cour=cour->suivant;   // Passe à @42
```



Entrée de liste

Entrée de liste

- Pour accéder au premier maillon d'une liste, il faut une variable d'entrée (ou poignée/handle) : c'est-à-dire une variable (par exemple du main), qui pointe sur le premier maillon.
- Comme T pour les tableaux, avec T qui est en fait un pointeur, cette variable est souvent appelé Liste, notée L.

• L peut être directement un pointeur :

```
1 maillon *L; // pointeur qui va \^etre affecté au premier maillon de la liste.
```

• L peut être une variable (ou un pointeur sur variable) de type struct dont l'un des champs pointe sur le premier maillon de la liste.
C'est ce qui sera fait dans les slides suivants).

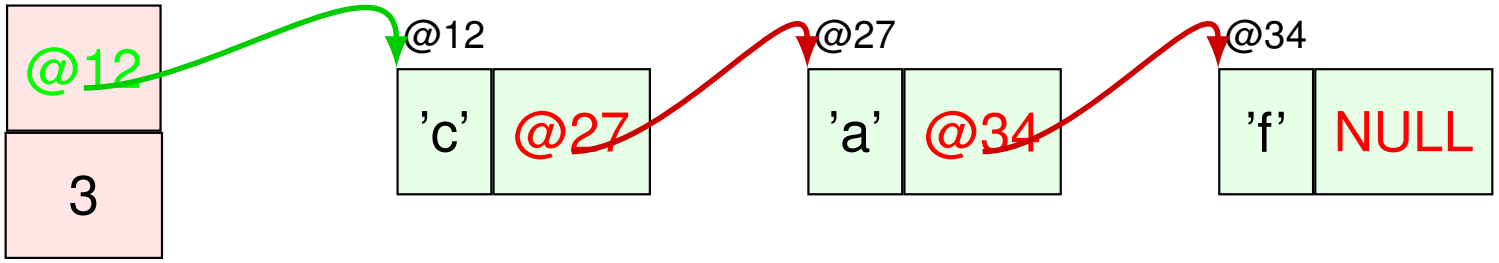
Liste simplément chaînée avec une entrée struct

Liste chaînée avec une entrée de type struct

On va considérer une entrée de liste de type struct contenant

- un pointeur sur le premier maillon (appelé aussi début ou tête de liste).
- un entier pour stocker la taille de la liste

```
1 typedef struct liste_s {
2   Maillon * debut;        /* pointeur vers le 1er maillon de la liste */
3   int taille;             /* nombre de maillons dans la liste */
4 } Liste;
```



Les listes chaînées

Méthodes de manipulation d'une liste

- Créer et Initialiser
- Détruire
- Tester si la liste est vide
- Connaître la taille de la liste
- Tester si un élément appartient à la liste
- Afficher la liste
- Accéder au i^e élément
- Ajouter un élément
- Extraire un élément

Les listes chaînées : création et initialisation

Initialisation

Une liste vide contient un pointeur de valeur NULL.

Cette fonction crée et initialise une liste vide puis renvoie l'adresse de la liste.

```
1 Liste * creer_liste_vide () {
2   Liste * res = malloc(sizeof(Liste));
3   res->debut = NULL;
4   res->taille = 0;
5   return res;
6 }
```

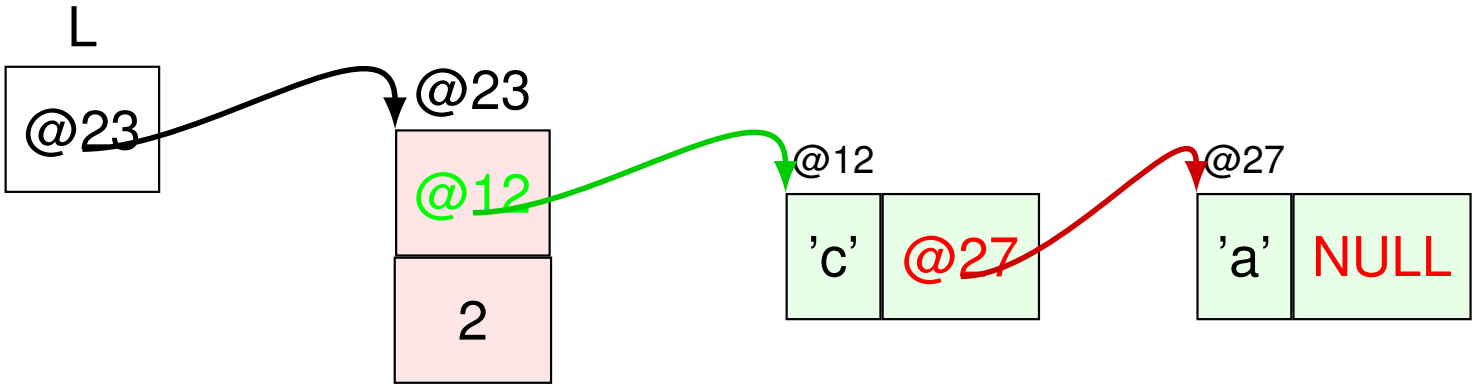
Complexité

- Temps : $O(1)$
- Espace : $O(1)$

Les listes chaînées : test de vacuité

Exemple de main

```
1 int main(){
2   Liste * L= creer_liste_vide();
3   ajouter_maillon_debut_liste (L, 'a');
4   ajouter_maillon_debut_liste (L, 'c');
5   afficher_liste(L);
6   detruire_liste(&L);
7   return 0;
8 }
```



Les listes chaînées : test de vacuité

Tester si une liste est vide

```
1 int test_liste_vide (Liste *l) {
2   return l->debut == NULL;
3 }
```

Complexité

- Temps : $O(1)$
- Espace : $O(1)$

Les listes chaînées : affichage

Affichage

- Comme avec un tableau on utilise une boucle pour parcourir la liste
- En revanche, la variable de boucle est un pointeur

```
1 void afficher_liste (Liste *l) {
2   Maillon * cour;
3   printf("( ");
4   for(cour = l->debut; cour != NULL; cour = cour->suivant)
5     printf("%c ", cour->valeur);
6   printf(") \n");
7 }
```

Complexité

- Temps : $O(n)$ où n est la taille de la liste
- Espace : $O(1)$

Les listes chaînées : affichage

Affichage

- Comme avec un tableau on utilise une boucle pour parcourir la liste
- En revanche, la variable de boucle est un pointeur

```
1 void afficher_liste (Liste *l) {
2   Maillon * cour;
3   printf("( ");
4   for(cour = l->debut; cour != NULL; cour = cour->suivant)
5     printf("%c ", cour->valeur);
6   printf(") \n");
7 }
```

Complexité

- Temps : $O(n)$ où n est la taille de la liste
- Espace : $O(1)$

Les listes chaînées : cardinalité (version 1)

Taille de la liste

```
1 unsigned int cardinal_liste1 (Liste *l) {
2   Maillon * cour;
3   unsigned int taille=0;
4   for(cour = l->debut; cour != NULL; cour = cour->suivant)
5     taille++;
6   return taille;
7 }
```

Complexité

- Temps : $O(n)$ où n est la taille de la liste
- Espace : $O(1)$

Les listes chaînées : cardinalité (version 2)

Taille de la liste

```
1 unsigned int cardinal_liste2 (Liste *l) {
2   return l->taille;
3 }
```

Complexité

- Temps : $O(1)$
- Espace : $O(1)$

Les listes chaînées : test d'appartenance

Teste si x est la valeur d'un Maillon de la Liste L

```
1 int test_appartenir_liste (Liste *l, char x) {
2   Maillon * cour = l->debut;
3   while (cour != NULL) {
4     if (cour->valeur == x)
5       return 1;
6     cour = cour->suivant;
7   }
8   return 0;
9 }
```

Complexité

- Temps : $O(n)$ où n est la taille de la liste
- Espace : $O(1)$

Les listes chaînées : test d'appartenance

Preuve de la fonction `test_appartenir_liste` :

Pour prouver cette fonction, il faut regarder les différents cas et exhiber qu'elle se comporte comme attendu dans chaque cas.

- Si la liste est vide, on n'entre pas dans la boucle et la fonction retourne Faux.
- Si x est dans un des maillons de la liste : la boucle avance jusqu'à trouver x dans un maillon et retourne alors Vrai.
- Si x n'est pas dans la liste, la boucle avance jusqu'à ce que *cour* soit NULL et retourne alors Faux. □

Les listes chaînées : accès en tête

Renvoie un pointeur vers le Maillon de début s'il existe et NULL sinon.

```
1 Maillon* acceder_debut_liste (Liste *l) {
2   return l->debut;
3 }
```

Complexité

- Temps : $O(1)$
- Espace : $O(1)$

Les listes chaînées : accès en position *i*

Accès au i^e élément

- Contrairement à la i^e clé d'un tableau, la i^e valeur d'une liste n'est pas directement accessible.
- exemple : pour accéder au 3e élément, il faut accéder au maillon **suivant** du maillon **suivant** du **début** de la liste.

Renvoie un pointeur vers le Maillon occupant la position `i` dans la Liste L si ce Maillon existe et NULL sinon.

```
1 Maillon * acceder_liste (Liste *l, unsigned int i) {
2   Maillon * cour=NULL;
3   for (cour = l->debut; cour != NULL && i != 0; i--)
4     cour = cour->suivant;
5   return cour;
6 }
```

Complexité

- Temps : $O(i)$
- Espace : $O(1)$

Les listes chaînées : accès en position *i*

Accès au i^e élément

- Contrairement à la i^e clé d'un tableau, la i^e valeur d'une liste n'est pas directement accessible.
- exemple : pour accéder au 3e élément, il faut accéder au maillon **suivant** du maillon **suivant** du **début** de la liste.

Renvoie un pointeur vers le Maillon occupant la position `i` dans la Liste L si ce Maillon existe et NULL sinon.

```
1 Maillon * acceder_liste (Liste *l, unsigned int i) {
2   Maillon * cour=NULL;
3   for (cour = l->debut; cour != NULL && i != 0; i--)
4     cour = cour->suivant;
5   return cour;
6 }
```

Complexité

- Temps : $O(i)$
- Espace : $O(1)$

Les listes chaînées : accès en position *i*

Preuve de la fonction `acceder_liste` :

Pour prouver cette fonction, il faut regarder les différents cas et exhiber qu'elle se comporte comme attendu dans chaque cas.

- Si la liste est vide, on n'entre pas dans la boucle et la fonction retourne NULL.
- Si la liste comporte plus de *i* éléments : la boucle avance de *i* maillons et retourne l'adresse du maillon *i*.
- Si la liste comporte moins de *i* éléments : la boucle avance jusqu'à ce que *cour* soit NULL et retourne alors NULL. □

Les listes chaînées : ajout d'un maillon

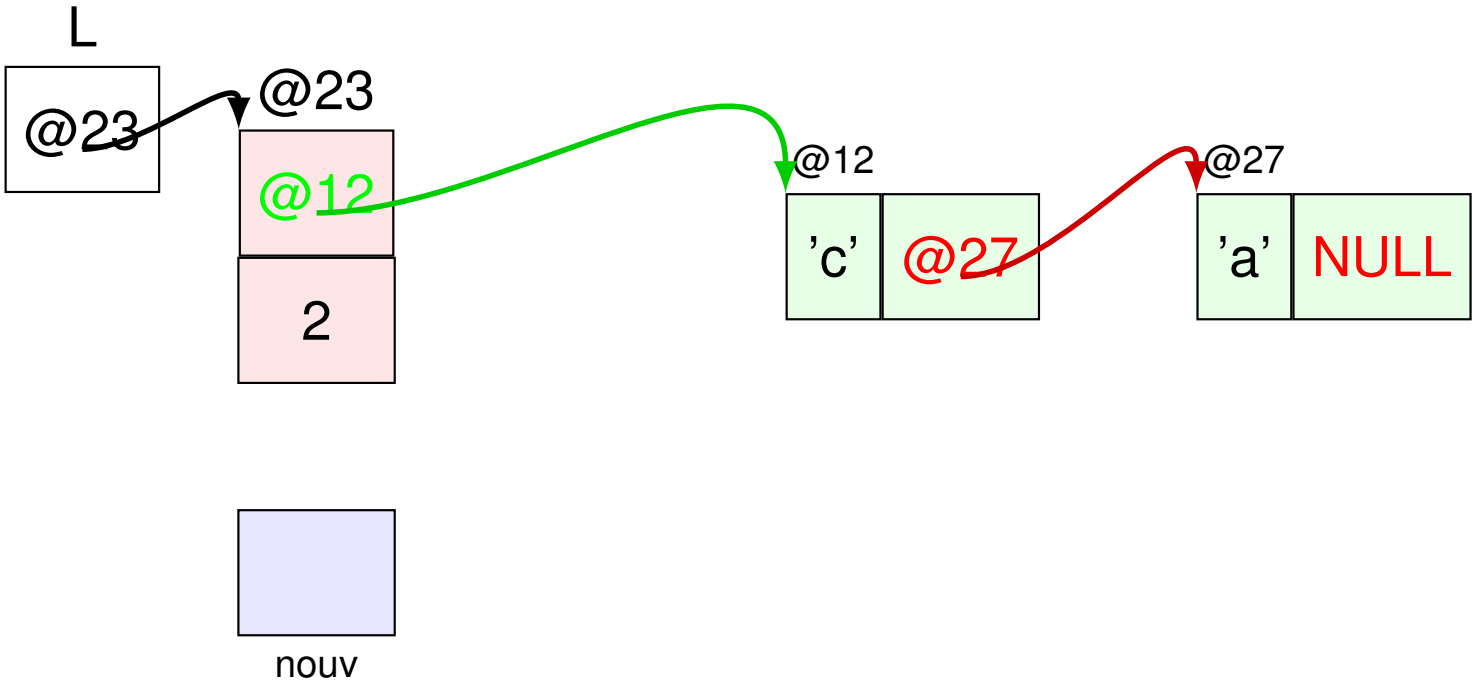
Ajout d'un maillon au début de la liste

Pour ajouter un nouveau maillon au début d'une liste :

(0) On crée un nouveau maillon

(1) On place le début de liste à la suite du nouveau maillon

(2) Le début de la liste après ajout pointe vers le nouveau maillon.



Les listes chaînées : ajout d'un maillon

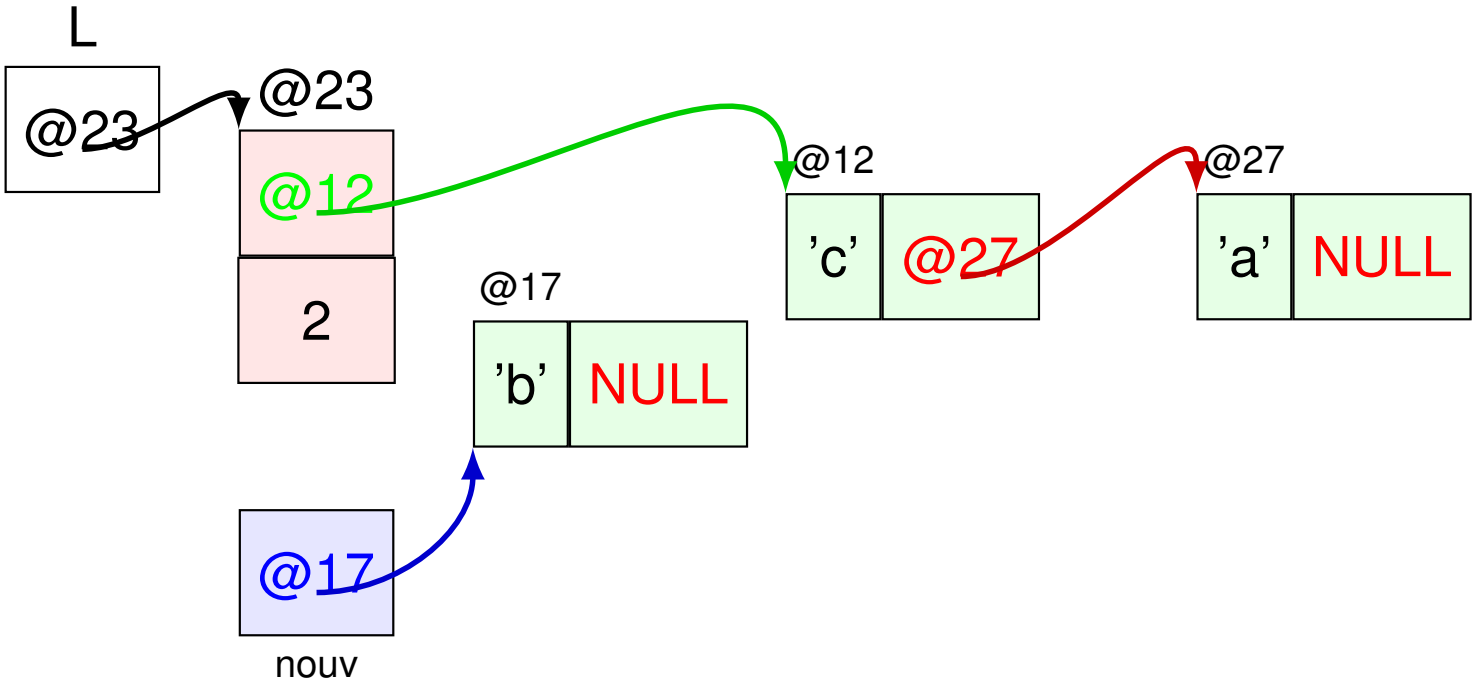
Ajout d'un maillon au début de la liste

Pour ajouter un nouveau maillon au début d'une liste :

(0) On crée un nouveau maillon

(1) On place le début de liste à la suite du nouveau maillon

(2) Le début de la liste après ajout pointe vers le nouveau maillon.



Les listes chaînées : ajout d'un maillon

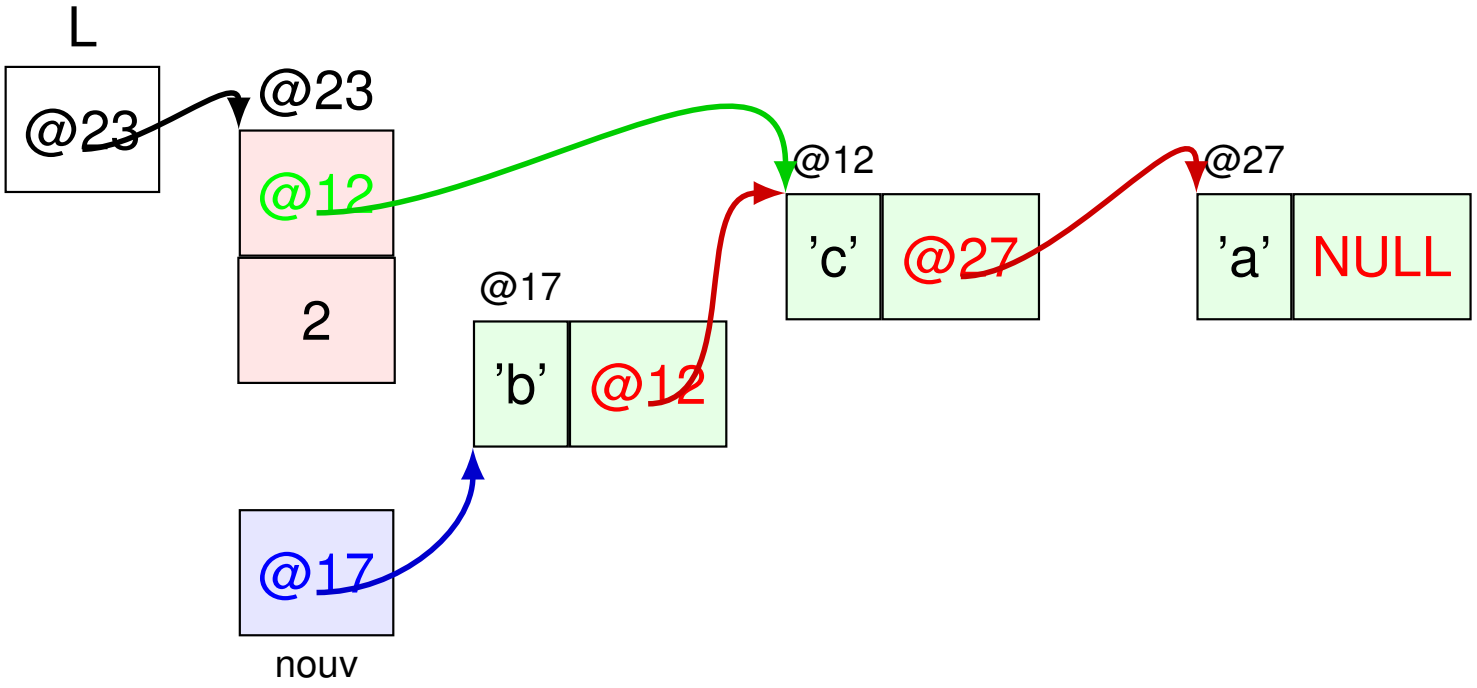
Ajout d'un maillon au début de la liste

Pour ajouter un nouveau maillon au début d'une liste :

(0) On crée un nouveau maillon

(1) On place le début de liste à la suite du nouveau maillon

(2) Le début de la liste après ajout pointe vers le nouveau maillon.



Les listes chaînées : ajout d'un maillon

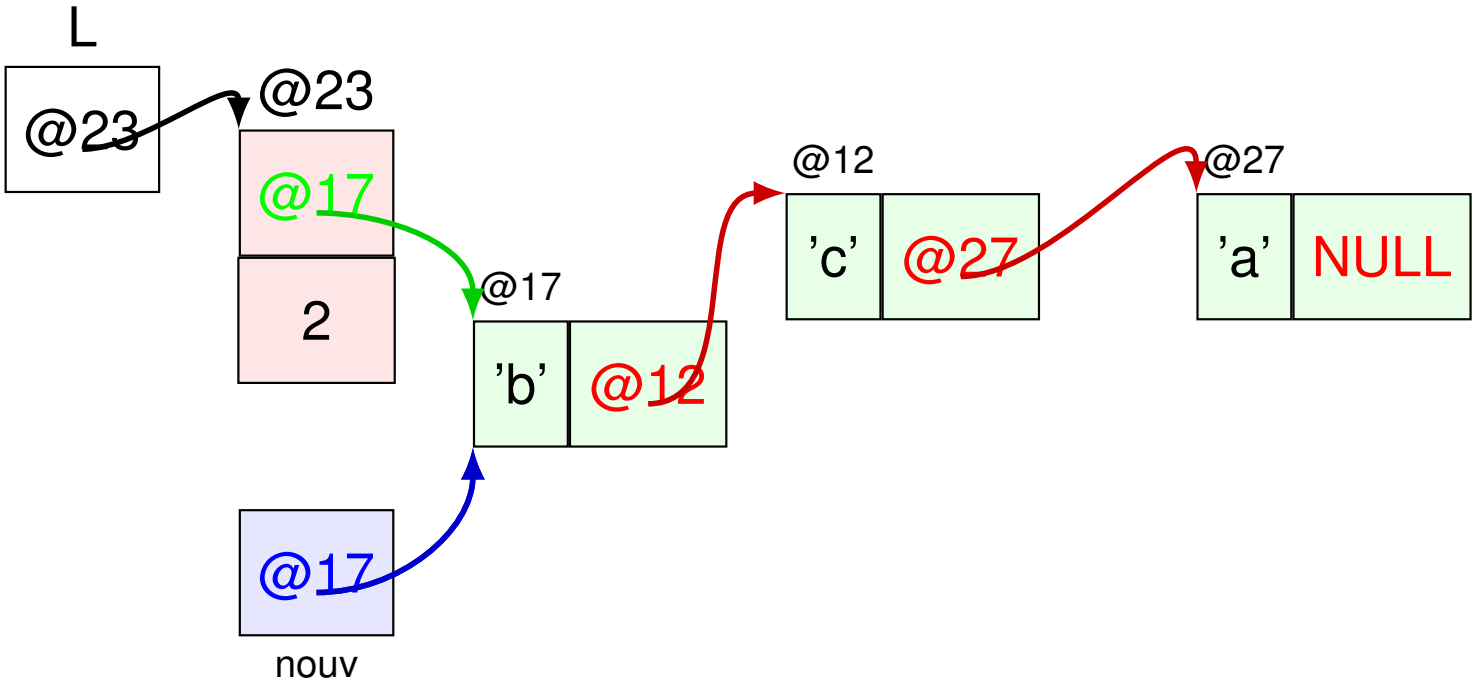
Ajout d'un maillon au début de la liste

Pour ajouter un nouveau maillon au début d'une liste :

(0) On crée un nouveau maillon

(1) On place le début de liste à la suite du nouveau maillon

(2) Le début de la liste après ajout pointe vers le nouveau maillon.



Les listes chaînées : ajout d'un maillon

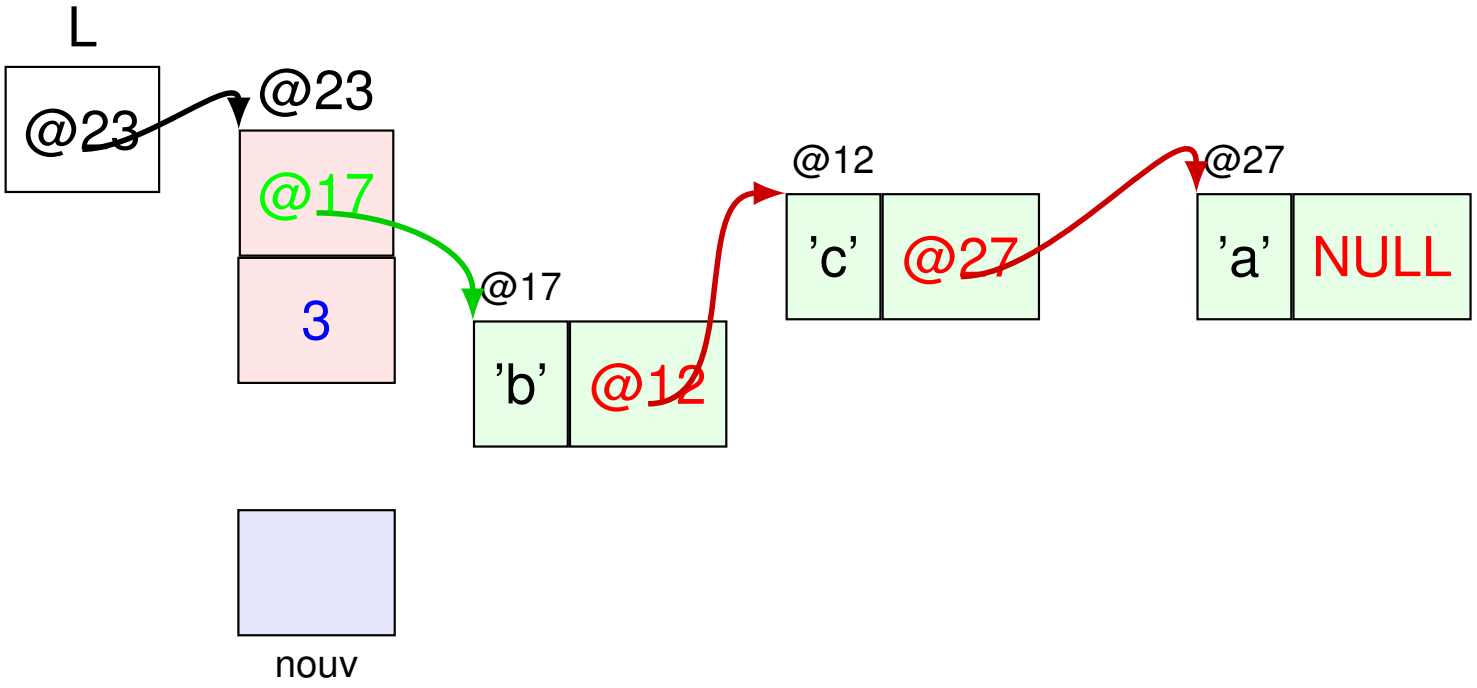
Ajout d'un maillon au début de la liste

Pour ajouter un nouveau maillon au début d'une liste :

(0) On crée un nouveau maillon

(1) On place le début de liste à la suite du nouveau maillon

(2) Le début de la liste après ajout pointe vers le nouveau maillon.



Les listes chaînées : ajout d'un élément

Ajoute un nouveau Maillon contenant *c* au début de la Liste pointée par *L*

```
1 void ajouter_maillon_debut_liste (Liste * l, char c) {
2   Maillon * nouv = creer_maillon(c);
3   nouv->suivant = l->debut;
4   l->debut = nouv;
5   l->taille++;
6 }
```

Complexité

- Temps : $O(1)$
- Espace : $O(1)$

Les listes chaînées : ajout d'un maillon

Preuve de la fonction `ajouter_maillon_debut_liste` :

Il n'y que deux cas à regarder

- Si la liste contient au moins un élément : le dessin que l'on a fait est à lui seul une preuve de validité
- Si la liste est vide : dans ce cas `L->debut` contient NULL
  et en ligne 3, `nouv->suivant` va donc valoir NULL.
  Ainsi, la liste contient un unique maillon.

A la fin de la fonction, le pointeur `nouv` est détruit avec la fin de la fonction, mais le maillon pointé reste en mémoire, pointée par L. □

Les listes chaînées : ajout en position i

Problème posé par l'accès

- L'accès au $(i-1)^e$ maillon a un coût linéaire.
- Une fois accédé à ce maillon, l'ajout d'un maillon à sa suite est en $O(1)$

Complexité

- Temps : $O(i)$
- Espace : $O(1)$

Remarque : pour un tableau, si la taille du tableau est suffisante, c'est en $O(n-i)$ car il faut décaler les $n-i$ cases en fin de tableau.

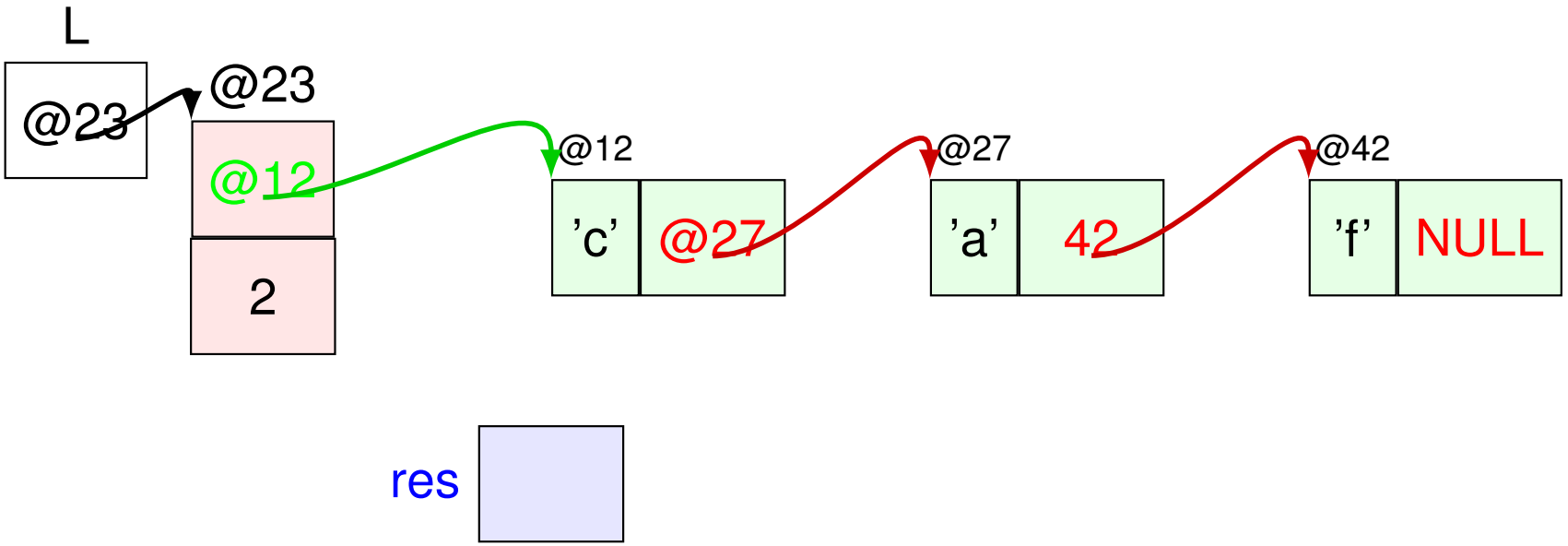
Les listes chaînées : extraire un élément

Extraire un maillon au début de la liste

(0) On stocke l'adresse du premier maillon dans une variable temporaire

(1) On fait pointer la liste sur le deuxième maillon

(2) On déconnecte le maillon de la liste



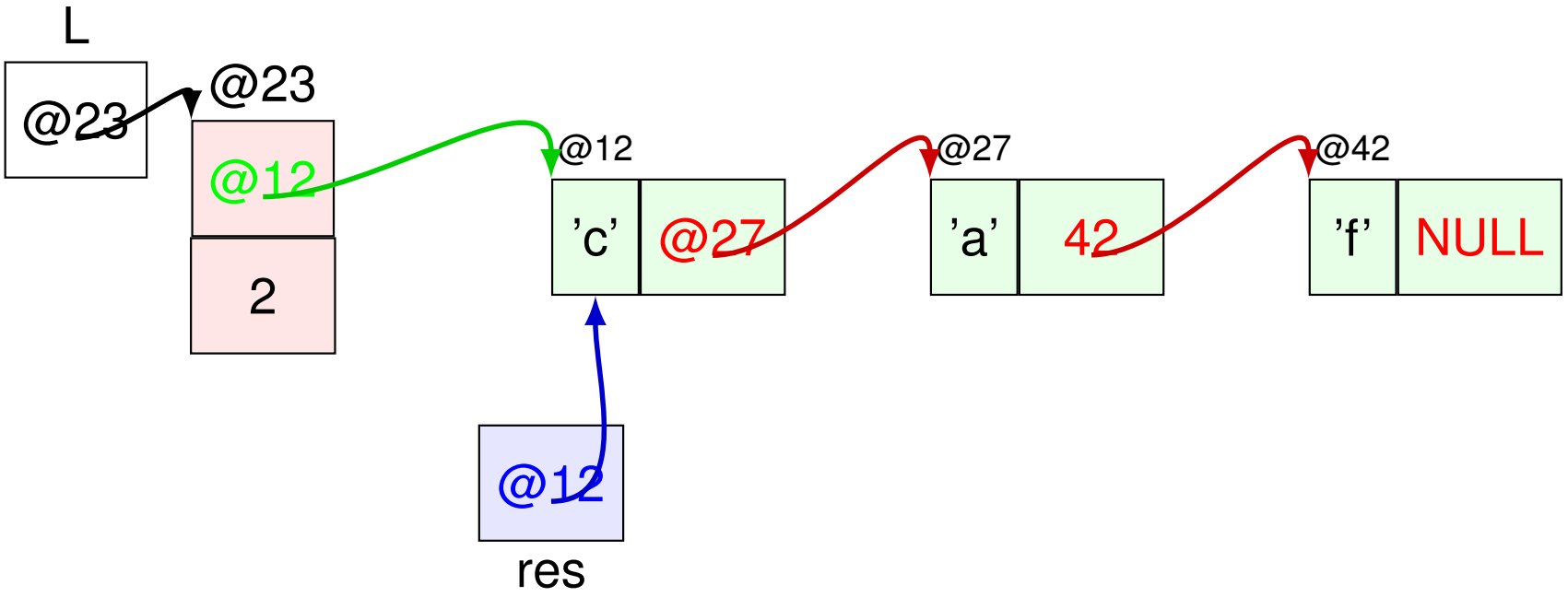
Les listes chaînées : extraire un élément

Extraire un maillon au début de la liste

(0) On stocke l'adresse du premier maillon dans une variable temporaire

(1) On fait pointer la liste sur le deuxième maillon

(2) On déconnecte le maillon de la liste



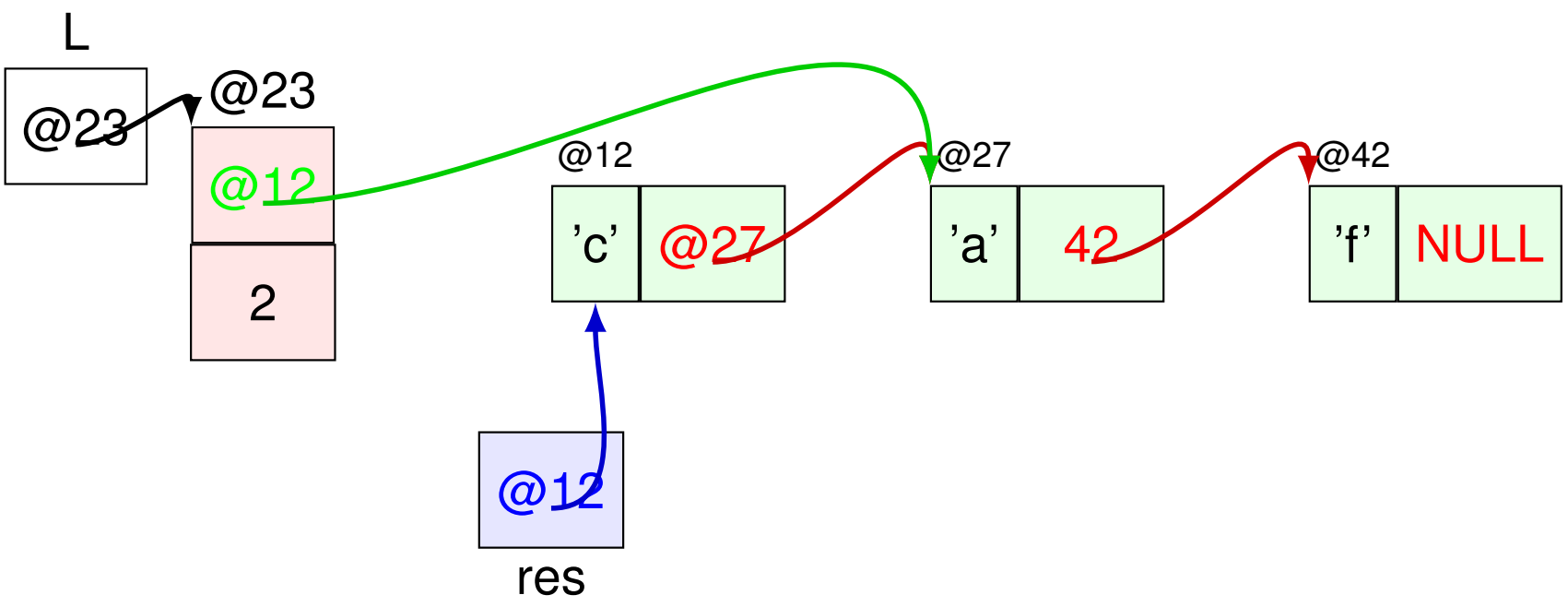
Les listes chaînées : extraire un élément

Extraire un maillon au début de la liste

(0) On stocke l'adresse du premier maillon dans une variable temporaire

(1) On fait pointer la liste sur le deuxième maillon

(2) On déconnecte le maillon de la liste



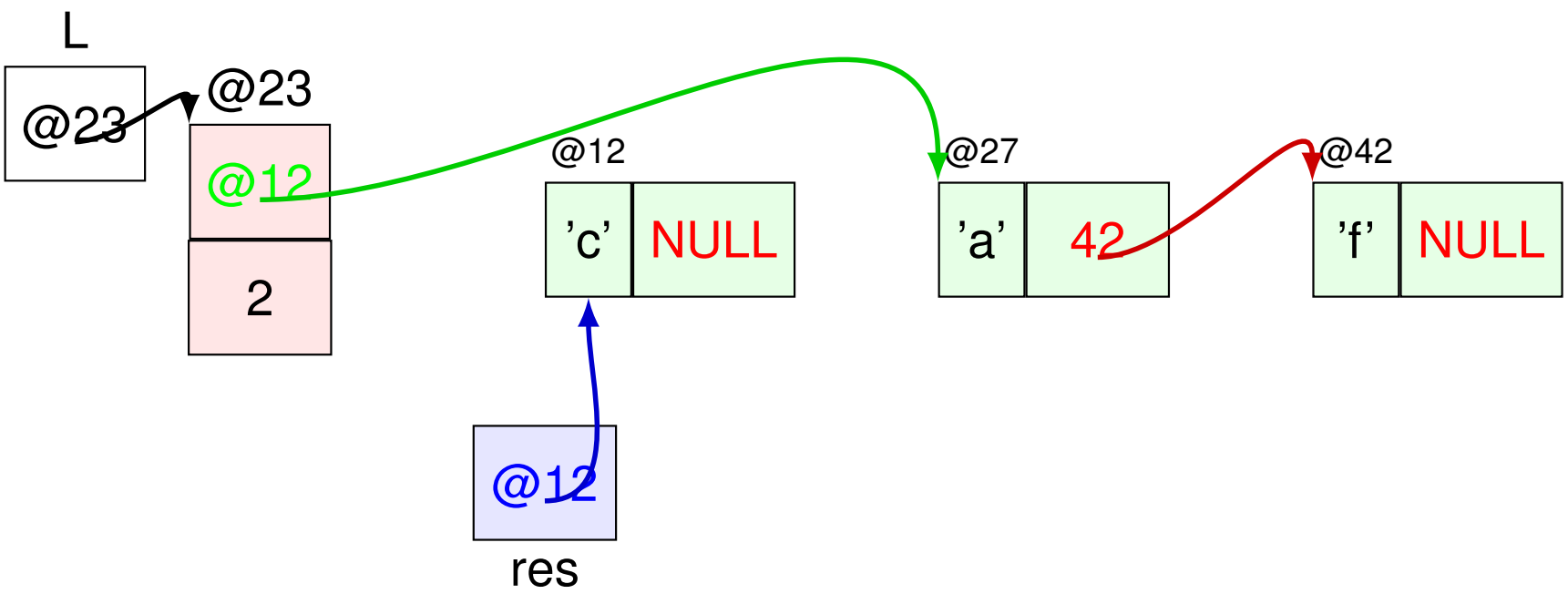
Les listes chaînées : extraire un élément

Extraire un maillon au début de la liste

(0) On stocke l'adresse du premier maillon dans une variable temporaire

(1) On fait pointer la liste sur le deuxième maillon

(2) On déconnecte le maillon de la liste



Les listes chaînées : extraire un élément

Extrait le Maillon qui se trouve au debut de la Liste pointée par L s'il existe. Renvoie un pointeur vers ce Maillon ou NULL sinon

```
1 Maillon * extraire_maillon_debut_liste (Liste * l) {
2   Maillon * res = l->debut;
3   if (res != NULL) {
4     l->debut = res->suivant;
5     l->taille --;
6     res->suivant = NULL;
7   }
8   return res;
9 }
```

Complexité

- Temps : $O(1)$
- Espace : $O(1)$

Les listes chaînées : extraire un maillon en début de liste

Preuve de la fonction `extraire_maillon_debut_liste` :

Il y a trois cas à considérer

- Si la liste est vide : la fonction ne fait rien. `l->debut` contient NULL et NULL est retourné.
- Si la liste contient au moins 2 éléments : le dessin et son animation prouve que le code contient bien les étapes dans l'ordre.
- Si la liste ne contient qu'un unique élément. Dans ce cas, en ligne 4, `res->suivant` est NULL, donc la liste devient liste vide. □

On peut remarquer qu'ici, on doit retourner le maillon extrait de la liste.
Si l'on veut supprimer le maillon, il faut alors finir par `free(res)`.

Les listes chaînées : détruire

Pour détruire une liste...

On libère la mémoire **maillon par maillon**.

```
1 void detruire_liste (Liste ** l) {
2   while (!test_liste_vide(*l))
3     detruire_maillon(extraire_maillon_debut_liste(*l));
4   free(*l);
5   *l=NULL;
```

Complexité

- Temps : $O(n)$ où n est la taille de la liste
- Espace : $O(1)$

Une interface utilisateur

Quelle interface pour une liste “utilisateur”

Les fonctions précédentes manipulent des maillons et des pointeurs : cela rend difficile la manipulation de liste et surtout cela demande à l’utilisateur de connaître tous les détails de code.

On peut préférer comme interface

- Définir dans le main une variable de type List (et non un pointeur).
- Il n’y a donc pas besoin d’allouer *L*, mais il faut l’initialiser
- Les fonctions passent en paramètres les valeurs à ajouter et retourne les contenus des éléments.

```
1 List L;
2 initialiser_liste_vide(&L);
3 ajouter_debut_liste(&L, 'f');
4 ajouter_debut_liste(&L, 'd');
5 char c=extraire_debut_liste(&L);
6 detruire(&L);
```

(voir les exercices de TD)

Une interface utilisateur

Quelle interface pour une liste “utilisateur”

Les fonctions précédentes manipulent des maillons et des pointeurs : cela rend difficile la manipulation de liste et surtout cela demande à l’utilisateur de connaître tous les détails de code.

On peut préférer comme interface

- Définir dans le main une variable de type List (et non un pointeur).
- Il n’y a donc pas besoin d’allouer L, mais il faut l’initialiser
- Les fonctions passent en paramètres les valeurs à ajouter et retourne les contenus des éléments.

```
1 List L;
2 initialiser_liste_vide(&L);
3 ajouter_debut_liste(&L, 'f');
4 ajouter_debut_liste(&L, 'd');
5 char c=extraire_debut_liste(&L);
6 detruire(&L);
```

(voir les exercices de TD)

Récapitulatif

Complexité-pire cas des méthodes manipulant des listes suivant si elles sont codées par un tableau ou une liste chaînée.

Fonctions	**Tableau**	**Liste chaînée**
Initialisation	$O(1)$	$O(1)$
Test vide	$O(1)$	$O(1)$
Cardinalité	$O(1)$	$O(1)$
Affichage	$O(n)$	$O(n)$
Test appartient	$O(n)$	$O(n)$
Accès i^e élément	$O(1)$	$O(i)$
Ajout début	$O(n)$	$O(1)$
Extraire 1er élément	$O(n)$	$O(1)$
Destruction	$O(1)$	$O(n)$

De plus, un tableau nécessite la définition a priori d'un grand espace contigu, tandis que la liste chaînée réserve sa mémoire à chaque ajout.

SDA Liste
Autres implémentations de listes chaînées

Il existe d'autres implémentations du SDA Liste par des Listes Chaînées. On choisit la meilleure implémentation en fonction de l'espace mémoire utilisé mais surtout en fonction du cas d'utilisation et de leurs vitesses.

Défaut de la liste simplement chaînée :
coût $\Theta(n)$ de l'accès à l'élément de fin.

Réponse : La liste doublement chaînée (ou liste symétrique) qui est une liste chaînée où chaque élément possède un pointeur sur l'élément suivant ET sur l'élément précédent.

L'accès à la fin de liste est au même coût que l'accès en tête de liste : $\Theta(1)$.

C'est l'implémentation choisie par la STL du C++ et par Java.

SDA Liste
Autres implémentations de listes chaînées

Il existe d'autres implémentations du SDA Liste par des Listes Chaînées. On choisit la meilleure implémentation en fonction de l'espace mémoire utilisé mais surtout en fonction du cas d'utilisation et de leurs vitesses.

Défaut de la liste simplement chaînée :
coût $\Theta(n)$ de l'accès à l'élément de fin.

Réponse : La liste doublement chaînée (ou liste symétrique) qui est une liste chaînée où chaque élément possède un pointeur sur l'élément suivant ET sur l'élément précédent.

L'accès à la fin de liste est au même coût que l'accès en tête de liste : $\Theta(1)$.

C'est l'implémentation choisie par la STL du C++ et par Java.

Autres implémentations de listes chaînées

Défaut de la liste simplement chaînée :

La recherche démarre toujours du même élément placé au début.

Réponse : La liste circulaire où l'élément de fin pointe sur l'élément de début. Ainsi, le pointeur `L` peut pointer sur un élément quelconque : en particulier le dernier visité.

Elle n'a pas d'avantage particulier en complexité pire-cas.

En revanche, elle permet d'effectuer la recherche à partir du dernier élément visité. Cela permet, dans certains cas, d'avoir un meilleur fonctionnement en moyenne.

Autres implémentations de listes chaînées

Défaut de la liste simplement chaînée :
La recherche démarre toujours du même élément placé au début.

Réponse : La liste circulaire où l'élément de fin pointe sur l'élément de début. Ainsi, le pointeur `L` peut pointer sur un élément quelconque : en particulier le dernier visité.

Elle n'a pas d'avantage particulier en complexité pire-cas.

En revanche, elle permet d'effectuer la recherche à partir du dernier élément visité. Cela permet, dans certains cas, d'avoir un meilleur fonctionnement en moyenne.

1 Structures de données abstraites (SDA)

2 SDA Liste

3 Implementation d'une liste par un Tableau

4 Implementation d'une liste par une Liste Simplement Chaînée

5 Pile et File
- SDA File
- SDA Pile
- Implémentation d'une pile ou d'une file
- Complexité

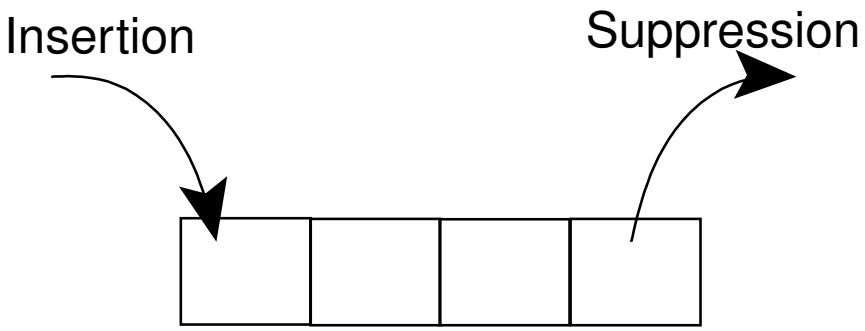
6 Pile d'éxecution et Dérécursification

Pile et File

On s'intéresse ici aux listes gérées par la politique
- LIFO (dernier arrivé, premier traité) : PILE
- FIFO (premier arrivé, premier traité) : FILE

File (Queue)

Une file est une structure de données représentant un ensemble de données unitaires (homogènes),
- permettant l'insertion aisée de nouvelles données
- permettant l'extraction aisée de la **plus ancienne donnée insérée.**



SDA File

- On l'appelle parfois "file d'attente"
- Son nom logique est "gestion FIFO" : First In, First Out
- Fréquemment utilisée en informatique :
  - File d'attente des exécutions
  - Représentation des activités d'une machine-outil
  - Compilation
  - "Pipe" sous linux
  - et tout usage algorithmique où l'on doit manipuler des données selon leur ordre d'arrivée FIFO

SDA File

Quelles opérations sur une file ?

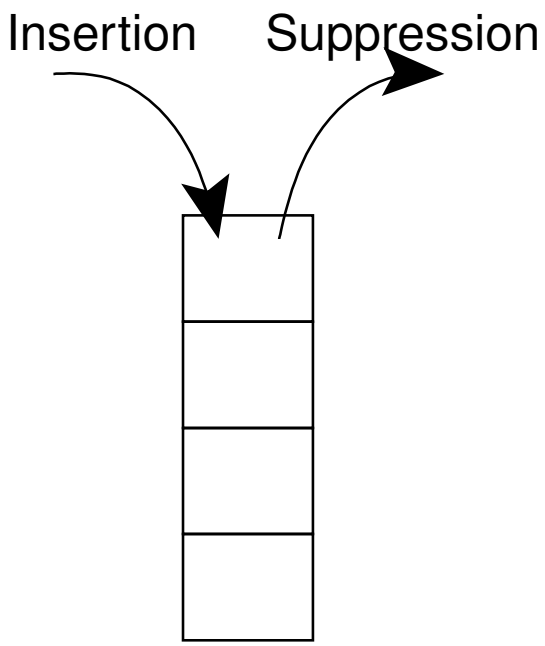
Opérations usuelles

- **Créer** une file vide : `creeFile(P)`
- **Tester** si une file est vide : `estFileVide(P)`
- **Insérer** un élément dans la file : `enfile(P,e)`
- **Supprimer** l'élément dans la file : `defile(P)`
- **Récupérer** l'élément en fin de file : `FinFile(P)`

SDA Pile

Pile (Stack)

Une pile est une structure de données représentant un ensemble de données unitaires (homogènes),
- permettant l'insertion aisée de nouvelles données
- permettant l'extraction aisée de la **dernière donnée insérée.**



SDA Pile

- On l'appelle parfois "structure en pile d'assiettes"
- Son nom logique est "gestion LIFO" : Last In, First Out
- Fréquemment utilisée en informatique :
  - Évaluation d'expression arithmétique
  - Pile d'exécution (pile d'appels des fonctions)
  - "Undo/Redo" dans un logiciel
  - et tout usage algorithmique où l'on doit manipuler des données selon leur ordre d'arrivée LIFO

SDA Pile

Quelles opérations sur une pile ?

Opérations usuelles

- **Créer** une pile vide : `creePile(P)`
- **Tester** si une pile est vide : `estPileVide(P)`
- **Empiler** un élément dans la pile : `empile(P,e)`
- **Dépiler** l'élément en tête de pile : `depile(P)`
- **Récupérer** l'élément en tête de pile : `tetePile(P)`

Implémentations

Implémenations par un tableau ou une liste chaînée

- Dans le cas où l'on peut déterminer à l'avance le nombre maximal d'éléments que peut contenir une pile, on peut l'implémenter par un tableau.
- Mais le cas général est justement celui d'une pile ou d'une file dont on ne connaît pas le nombre d'éléments à l'avance : on préfère utiliser une liste chaînée.
  - Pour une pile : une liste simplement chaînée avec insertion et suppression en tête
  - Pour une file : soit une liste doublement chaînée (voir TD), soit une liste simplement chaînée avec un pointeur en tête et un pointeur en fin.

Complexité des opérations Pile ou File

- `creePile(P)`

- Implémentation Liste Chaînée : $\Theta(1)$ / Mémoire vide
- Implémentation Tableau : $\Theta(1)$ / Un tableau de taille TailleMax

- `empile(P,e)`

- Implémentation Liste Chaînée : $\Theta(1)$ / Alloue un element
- Implémentation Tableau : $\Theta(1)$ ou $\Theta(n)$ s'il faut allonger le pile

- `depile(P)`

- Implémentation Liste Chaînée : $\Theta(1)$
- Implémentation Tableau : $\Theta(1)$

- `tetePile(P)`

- Implémentation Liste Chaînée : $\Theta(1)$
- Implémentation Tableau : $\Theta(1)$

La différence est aussi sur la réservation mémoire !

1 Structures de données abstraites (SDA)

2 SDA Liste

3 Implementation d'une liste par un Tableau

4 Implementation d'une liste par une Liste Simplement Chaînée

5 Pile et File

6 Pile d'éxecution et Dérécursification

La Pile d'exécution

Considérons trois fonction `f, h` et leurs appels en cascade.gne appelant la fonction `f` intervient dans le programme.

```
1 int h(int x){
2   return x-4;
3 }
4
5 int g(int x){
6   return h(x)+1;
7 }
8
9 int f(int x){
0   return g(x)*2;
1 }
2
3 int main(){
4   printf("%d %d\n",f(5), g(3));
5   return 0;
6 }
```

On peut numéroter les appels aux fonctions dans l'ordre où les appels interviennent, ainsi que les valeurs passées en paramètres.

La Pile d'exécution

En pratique, chaque appel à une même fonction est indépendant des autres.

Lors d'un appel, une zone mémoire est réservée pour effectuer cet appel. Cett e zone mémoire contient les variables déclarées dans la fonction ainsi que les variables déclarées en paramètres.

La zone mémoire réservée par un appel n'est libérée que lorsque l'accolade fermante ou la commande `return` sont atteintes.

Donc lors d'un appel à une fonction `f` qui lance une fonction `g`, la zone mémoire de `f` reste en place pendant l'exécution de `g`.
De plus, une fois que la fonction `g` est terminée, l'exécution de `f` reprend.

Notons $D(f_i)$ le début du $i^{\text{ème}}$ appel à la fonction `f` et $F(f_i)$ sa fin.
Le programme précédent exécute dans l'ordre :
$D(f_1), D(g_1), D(h_1), F(h_1), F(g_1), F(f_1), D(g_2), D(h_2), F(h_2), F(g_2)$.

La Pile d'exécution

En pratique, chaque appel à une même fonction est indépendant des autres.

Lors d'un appel, une zone mémoire est réservée pour effectuer cet appel.
Cett e zone mémoire contient les variables déclarées dans la fonction ainsi que les variables déclarées en paramètres.

La zone mémoire réservée par un appel n'est libérée que lorsque l'accolade fermante ou la commande `return` sont atteintes.

Donc lors d'un appel à une fonction `f` qui lance une fonction `g`, la zone mémoire de `f` reste en place pendant l'exécution de `g`.
De plus, une fois que la fonction `g` est terminée, l'exécution de `f` reprend.

Notons $D(f_i)$ le début du $i^{\text{ème}}$ appel à la fonction `f` et $F(f_i)$ sa fin.
Le programme précédent exécute dans l'ordre :
$D(f_1), D(g_1), D(h_1), F(h_1), F(g_1), F(f_1), D(g_2), D(h_2), F(h_2), F(g_2)$.

La Pile d'exécution

Le stockage et l'ordre des exécutions des fonctions est gérées par une pile que l'on appelle la pile d'exécution.

Les appels s'empilent dans l'ordre de leur appel et se dépilent dans l'ordre de leur terminaison.

La pile contient les valeurs des paramètres effectifs, puis les valeurs des retours des fonctions.

Entre temps, la mémoire n'est donc pas libérée !
Cela peut poser problème s'il y a beaucoup d'appels imbriqués, comme c'est le cas pour des appels récursifs.

La Pile d'exécution

Le stockage et l'ordre des exécutions des fonctions est gérées par une pile que l'on appelle la pile d'exécution.

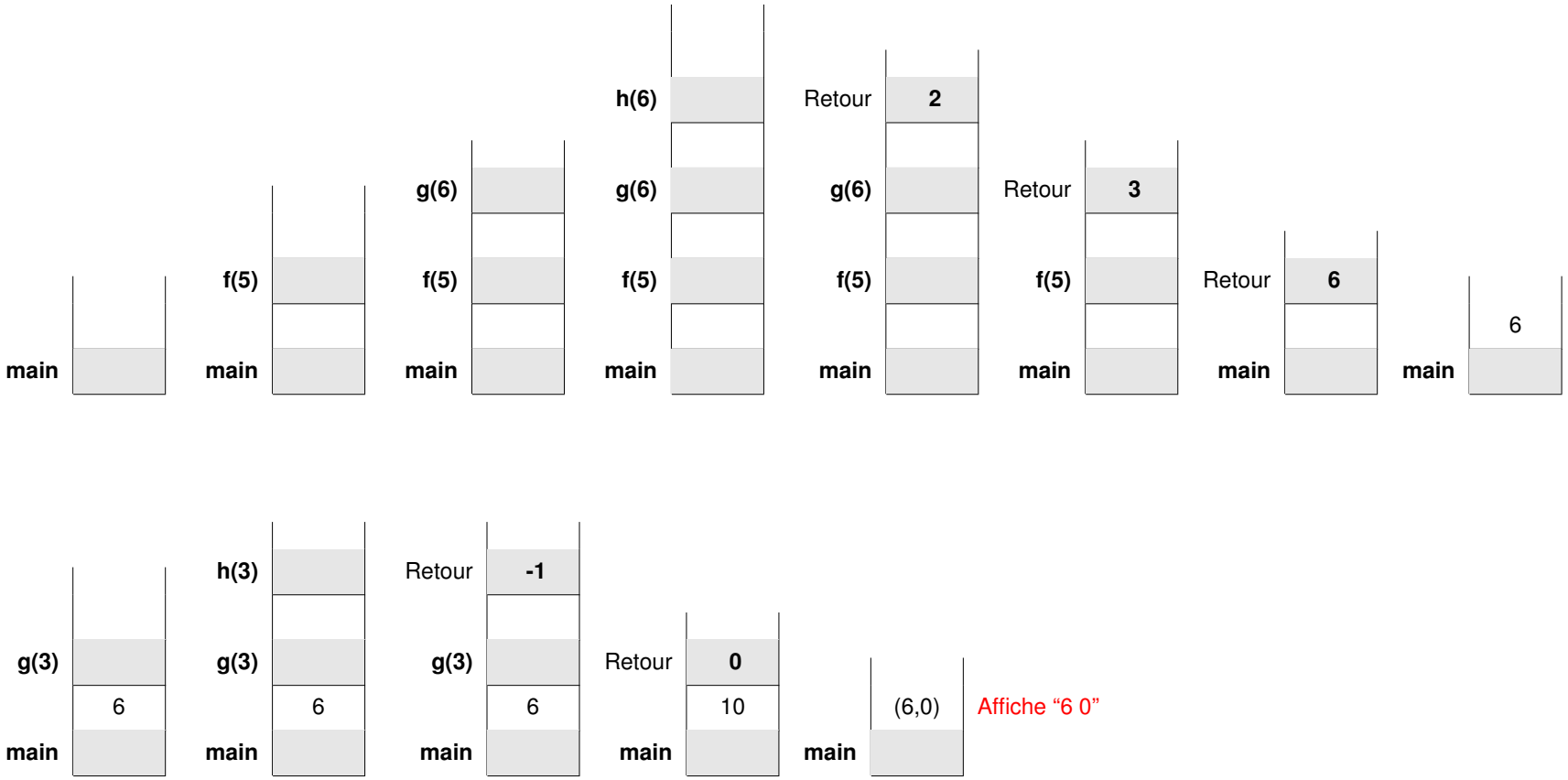
Les appels s'empilent dans l'ordre de leur appel et se dépilent dans l'ordre de leur terminaison.

La pile contient les valeurs des paramètres effectifs, puis les valeurs des retours des fonctions.

Entre temps, la mémoire n'est donc pas libérée !
Cela peut poser problème s'il y a beaucoup d'appels imbriqués, comme c'est le cas pour des appels récursifs.

Piles d'appels

Pile d'appels lors des appels :



Récursivité

On appelle récursivité le fait qu'une fonction contienne un appel elle-même.

Exemple : cette fonction `somme` prend un entier naturel en paramètre et calculant la somme des $n+1$ premiers entiers naturels.

```
1 int somme(int n){
2   if(n == 0){
3     return 0;
4   }
5   else{
6     return n + somme(n-1);
7   }
8 }
```

Ici la fonction va donc empiler n appels à la fonction `somme` dans la pile d'exécution :
$D(somme_4), D(somme_3), D(somme_2), D(somme_1), D(somme_0), F(somme_0)$
$F(somme_1), F(somme_2), F(somme_3), F(somme_4)$

Comme on l'a vu, la récursivité utilise de la mémoire en grande quantité car les fonction appelée conserve une place en mémoire jusqu'à leur fin de vie. Et dans cet exemple, c'est donc à la fin du programme que se libère la pile d'exécution.

Récursivité

Exemple : La suite de Fibonacci :

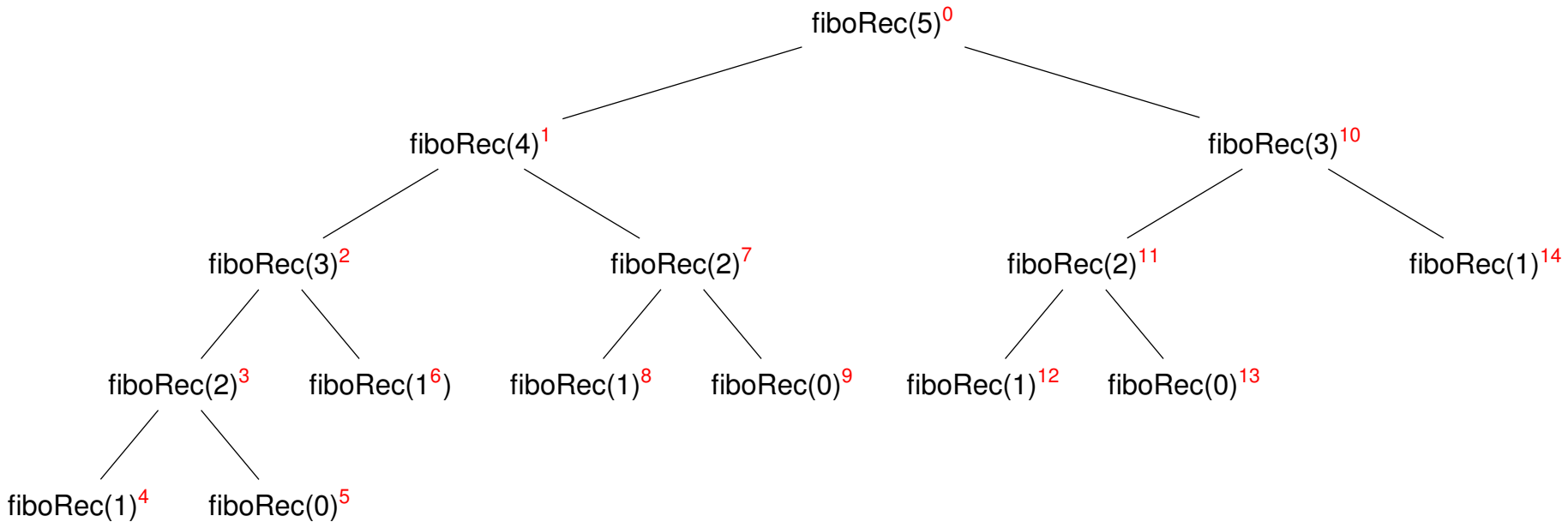
$$u_n = \begin{cases} u_{n-1} + u_{n-2} & si \quad n \geq 2 \\ 1 & si \quad n = 1 \\ 0 & si \quad n = 0 \end{cases}$$

Son code naturellement récursif

```
1 int FiboRec(int n){
2     if ( (n == 0) || (n == 1) )  return n;
3     return FiboRec(n-2) + FiboRec(n-1);
4 }
```

Autre exemple : la suite de fibonnacci

Arborescence des appels récursifs



L'exposant en rouge désigne l'ordre dans lequel les appels sont réalisés.
Cette arborescence représente quels sont les appels faits par un appel donné.

C'est un arbre binaire presque complet : il y a $O(2^n)$ appels récursifs.
Mais ils ne sont pas tous en même temps dans la pile.
Ici la pile est remplie dans l'ordre d'exploration de l'arbre : au pire, la pile comporte n appels.

Récursivité terminale

Une fonction est récursive terminale si l'appel récursif est directement retourné par la fonction appelante.
Le retour de la fonction ne doit donc pas être le résultat d'une opération utilisant le retour de l'appel récursif.

On essaie, lorsque cela est possible (ce n'est pas toujours le cas) de réaliser une récursivité terminale.
La récursivité terminale permet de limiter les appels récursifs qui sont stockés dans la pile d'exécution du programme.

Voir le cours de Programmation Impérative 2 (non au programme pour ce semestre)

```
1 int somme(int n, int accu){
2   if(n == 0){
3     return accu;
4   }
5   else{
6     return somme(n-1, n+accu);
7   }
8 }
```

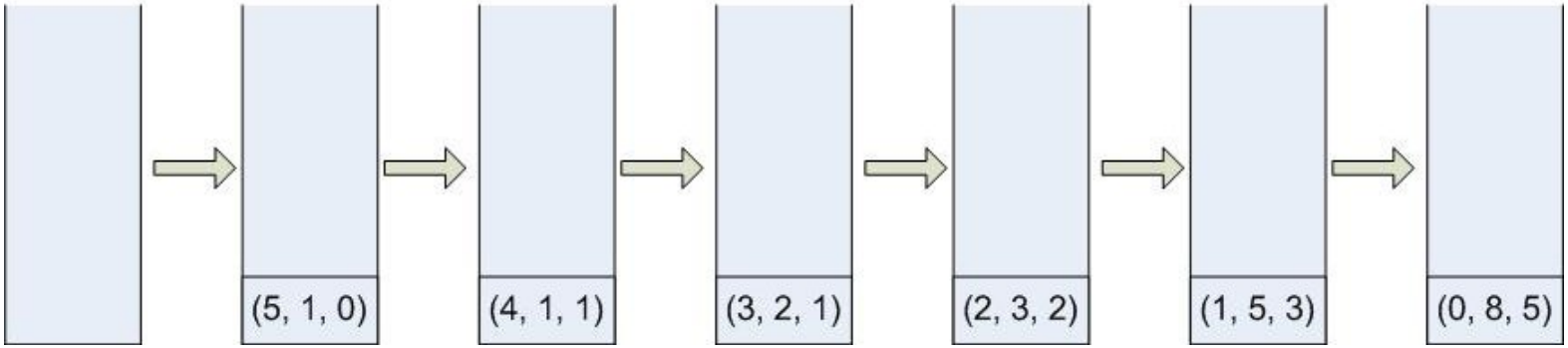
Il faut compiler avec l'option `-O2` de gcc.

Récursivité terminale et mémoïsation

On peut le faire également pour Fibonacci... en se creusant le crâne...

```
1 int FiboRecTerm(int n, long int acc1, long int acc2){
2   if (n==0) return acc1;
3   if (n==1) return acc2;
4   return FiboRecTerm(n-1, acc2, acc1+acc2);
5 }
```

Dans ce cas de récursivité terminale, la taille de la pile d'exécution des appels reste constante.



Mais la complexité reste en $O(2^n)$!!!

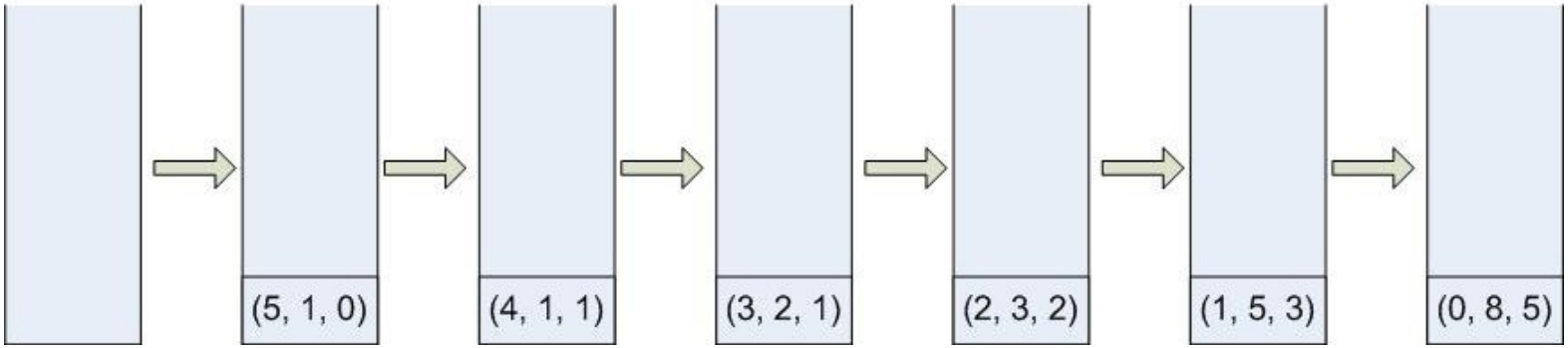
Il faut en plus utiliser la mémoïsation (voir cours Programmation Impérative 2) pour réduire la complexité à $O(n)$!

Récursivité terminale et mémoïsation

On peut le faire également pour Fibonacci... en se creusant le crâne...

```
1 int FiboRecTerm(int n, long int acc1, long int acc2){
2   if (n==0) return acc1;
3   if (n==1) return acc2;
4   return FiboRecTerm(n-1, acc2, acc1+acc2);
5 }
```

Dans ce cas de récursivité terminale, la taille de la pile d'exécution des appels reste constante.



Mais la complexité reste en $O(2^n)$!!!

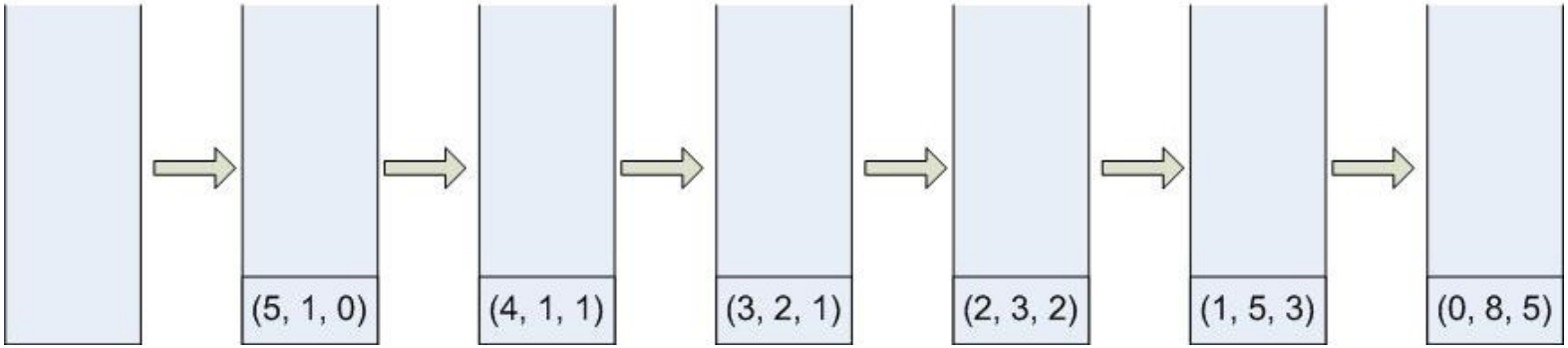
Il faut en plus utiliser la mémoïsation (voir cours Programmation Impérative 2) pour réduire la complexité à $O(n)$!

Récursivité terminale et mémoïsation

On peut le faire également pour Fibonacci... en se creusant le crâne...

```
1 int FiboRecTerm(int n, long int acc1, long int acc2){
2   if (n==0) return acc1;
3   if (n==1) return acc2;
4   return FiboRecTerm(n-1, acc2, acc1+acc2);
5 }
```

Dans ce cas de récursivité terminale, la taille de la pile d'exécution des appels reste constante.



Mais la complexité reste en $O(2^n)$!!!

Il faut en plus utiliser la mémoïsation (voir cours Programmation Impérative 2) pour réduire la complexité à $O(n)$!

Dérécursification

On appelle Dérécursifier le fait de donner une version non récursive d'un code récursif.

On peut toujours le faire en utilisant une pile qui va reproduire le fonctionnement de la pile d'exécution des appels des fonctions

Pour cela, on peut par exemple considérer

- P : pile permettant d'empiler (D, n) pour les débuts et (F, n) pour les fins d'appels pour une valeur n.
- Q : pile des valeurs retours

On simule le système des piles d'appels par une boucle qui s'arrête quand il n'y a plus d'appels à traiter.

A chaque itération, on dépile le premier élément qui est soit
- un début : alors on lancer les appels suivants
- une fin : la pile des retours contient les valeurs à utiliser pour le calcul

Dérécursification

On appelle Dérécursifier le fait de donner une version non récursive d'un code récursif.

On peut toujours le faire en utilisant une pile qui va reproduire le fonctionnement de la pile d'exécution des appels des fonctions

Pour cela, on peut par exemple considérer

- P : pile permettant d'empiler (D, n) pour les débuts et (F, n) pour les fins d'appels pour une valeur n.
- Q : pile des valeurs retours

On simule le système des piles d'appels par une boucle qui s'arrête quand il n'y a plus d'appels à traiter.

A chaque itération, on dépile le premier élément qui est soit
- un début : alors on lancer les appels suivants
- une fin : la pile des retours contient les valeurs à utiliser pour le calcul

Dérécursification

```
1  int FiboPile (int n) {
2     int res = 0;
3     appel a;
4     Pile_appel *P = creePileVide_appel(); // Pile des appels
5     Pile *Q = creePileVide();             // Pile des valeurs retour
6
7     empile_appel (P, 'D', n);             // Empile (D,n) l'appel initial
8
9     while (!estPileVide_appel (*P)) {
0        a = depile_appel (P);
1
2        if (a.type=='D'){
3           if (a.num == 0) empile(Q, 0);
4           if (a.num == 1) empile(Q, 1);
5           if (a.num >= 2){
6              empile_appel (P, 'F', a.num);
7              empile_appel (P, 'D', a.num - 2);
8              empile_appel (P, 'D', a.num - 1);
9           }
20       }
21       if (a.type=='F')
22          empile(Q, depile(Q) + depile(Q));
23    }
24    free(P);
25    res=depile(Q);
26    free(Q);
27    return res;
28 }
```

```
P: [ (D, 5) ]
Q: []
P: [ (F, 5)  (D, 3)  (D, 4) ]
Q: []
P: [ (F, 5)  (D, 3)  (F, 4)  (D, 2)  (D, 3) ]
Q: []
P: [ (F, 5)  (D, 3)  (F, 4)  (D, 2)  (F, 3)  (D, 1)  (D, 2) ]
Q: []
P: [ (F, 5)  (D, 3)  (F, 4)  (D, 2)  (F, 3)  (D, 1)  (F, 2)  (D, 0)  (D, 1) ]
Q: []
P: [ (F, 5)  (D, 3)  (F, 4)  (D, 2)  (F, 3)  (D, 1)  (F, 2)  (D, 0) ]
Q: [ 1 ]
P: [ (F, 5)  (D, 3)  (F, 4)  (D, 2)  (F, 3)  (D, 1)  (F, 2) ]
Q: [ 1  0 ]
P: [ (F, 5)  (D, 3)  (F, 4)  (D, 2)  (F, 3)  (D, 1) ]
Q: [ 1 ]
P: [ (F, 5)  (D, 3)  (F, 4)  (D, 2)  (F, 3) ]
Q: [ 1  1 ]
P: [ (F, 5)  (D, 3)  (F, 4)  (D, 2) ]
Q: [ 2 ]
P: [ (F, 5)  (D, 3)  (F, 4)  (F, 2)  (D, 0)  (D, 1) ]
Q: [ 2 ]
P: [ (F, 5)  (D, 3)  (F, 4)  (F, 2)  (D, 0) ]
Q: [ 2  1 ]
P: [ (F, 5)  (D, 3)  (F, 4)  (F, 2) ]
Q: [ 2  1  0 ]
P: [ (F, 5)  (D, 3)  (F, 4) ]
Q: [ 2  1 ]
P: [ (F, 5)  (D, 3) ]
Q: [ 3 ]
P: [ (F, 5)  (F, 3)  (D, 1)  (D, 2) ]
Q: [ 3 ]
P: [ (F, 5)  (F, 3)  (D, 1)  (F, 2)  (D, 0)  (D, 1) ]
Q: [ 3 ]
P: [ (F, 5)  (F, 3)  (D, 1)  (F, 2)  (D, 0) ]
Q: [ 3  1 ]
P: [ (F, 5)  (F, 3)  (D, 1)  (F, 2) ]
Q: [ 3  1  0 ]
P: [ (F, 5)  (F, 3)  (D, 1) ]
Q: [ 3  1 ]
P: [ (F, 5)  (F, 3) ]
Q: [ 3  1  1 ]
P: [ (F, 5) ]
Q: [ 3  2 ]   et retourne 5
```

Récursivité et Dérécursivité

La dérécursification ne fait pas gagner d'itérations et de ne réduit pas la complexité-mémoire.

Mais elle permet de réflechir à optimiser la mémoire utiliser pour une fonction. En effet, on peut limiter la taille prise par chaque appel.

Et cela permet aussi de réfléchir à la structure de l'appels récursifs...

Récursivité et Dérécursivité

La dérécursification ne fait pas gagner d'itérations et de ne réduit pas la complexité-mémoire.

Mais elle permet de réflechir à optimiser la mémoire utiliser pour une fonction. En effet, on peut limiter la taille prise par chaque appel.

Et cela permet aussi de réfléchir à la structure de l'appels récursifs...

Récursivité et Dérécursivité

... en effet, ici, en “utilisant notre intelligence humaine”, on peut quand même se rendre compte que on peut faire beaucoup mieux !

Il suffit de voir que les appels de la fonction Fibonnacci ne sont pas “croisés” : ils sont en fait linéaire mais à deux niveaux.
On peut donc faire circuler les valeurs des valeurs précédentes et les additionner :

```
1 int FiboIter(int n){
2   long int nMoins1 = 1, nMoins2 = 0, res = 1;
3   int i;
4   if ( (n == 0) || (n == 1) ) return n;
5   for( i=2; i<=n; i++){
6         res  = nMoins1 + nMoins2;
7         nMoins2 = nMoins1;
8         nMoins1 = res;
9   }
0   return res;
1 }
```

C’est la fonction la plus rapide pour le calcul de la suite de Fibonacci : il est en $O(n)$ avec une occupation mémoire en $O(1)$.
Malheureusement, on ne peut pas toujours réduire le temps de cacul et l’occupation mémoire à ce point !

Récursivité et Dérécursivité

... en effet, ici, en “utilisant notre intelligence humaine”, on peut quand même se rendre compte que on peut faire beaucoup mieux !

Il suffit de voir que les appels de la fonction Fibonnacci ne sont pas “croisés” : ils sont en fait linéaire mais à deux niveaux.
On peut donc faire circuler les valeurs des valeurs précédentes et les additionner :

```
1 int FiboIter(int n){
2   long int nMoins1 = 1, nMoins2 = 0, res = 1;
3   int i;
4   if ( (n == 0) || (n == 1) ) return n;
5   for( i=2; i<=n; i++){
6         res  = nMoins1 + nMoins2;
7         nMoins2 = nMoins1;
8         nMoins1 = res;
9   }
0   return res;
1 }
```

C’est la fonction la plus rapide pour le calcul de la suite de Fibonacci : il est en $O(n)$ avec une occupation mémoire en $O(1)$.
Malheureusement, on ne peut pas toujours réduire le temps de cacul et l’occupation mémoire à ce point !